

Cyber Attacks Prevention Toward Prosumer-Based EV Charging Stations: An Edge-Assisted Federated Prototype Knowledge Distillation Approach

Luyao Zou¹, Quang Hieu Vo¹, Kitae Kim¹, *Graduate Student Member, IEEE*, Huy Q. Le¹,
Chu Myaet Thwal¹, Chaoning Zhang¹, *Senior Member, IEEE*, and Choong Seon Hong¹, *Fellow, IEEE*

Abstract—In this paper, cyber-attack prevention for the prosumer-based electric vehicle (EV) charging stations (EVCSSs) is investigated, which covers two aspects: 1) cyber-attack detection on prosumers' network traffic (NT) data, and 2) cyber-attack intervention. To establish an effective prevention mechanism, several challenges need to be tackled, for instance, the NT data per prosumer may be non-independent and identically distributed (non-IID), and the boundary between benign and malicious traffic becomes blurred. To this end, we propose an edge-assisted federated prototype knowledge distillation (E-FPKD) approach, where each client is deployed on a dedicated local edge server (DLES) and can report its availability for joining the federated learning (FL) process. Prior to the E-FPKD approach, to enhance accuracy, the Pearson Correlation Coefficient is adopted for feature selection. Regarding the proposed E-FPKD approach, we integrate the knowledge distillation and prototype aggregation technique into FL to deal with the non-IID challenge. To address the boundary issue, instead of directly calculating the distance between benign and malicious traffic, we consider maximizing the overall detection correctness of all prosumers (ODC), which can mitigate the computational cost compared with the former way. After detection, a rule-based method will be triggered at each DLES for cyber-attack intervention. Experimental analysis demonstrates that the proposed E-FPKD can achieve the largest ODC on NSL-KDD, UNSW-NB15, and IoTID20 datasets in both binary and multi-class classification, compared with

baselines. For instance, the ODC for IoTID20 obtained via the proposed method is separately 0.3782% and 4.4471% greater than FedProto and FedAU in multi-class classification.

Index Terms—Cyber-attack prevention, prosumer-based EV charging stations, non-IID data, federated prototype knowledge distillation.

I. INTRODUCTION

AT PRESENT, electric vehicles (EVs) have gradually penetrated into our daily lives, which have the potential to reduce the emissions of greenhouse gas thereby protecting the environment [1]. Specifically, there may be around 125 million EVs being expected to be on the road by 2030 [2]. By 2040, EVs will account for 55% of the new global vehicle sales and 33% of vehicles on the road estimated by Bloomberg [3]. Owing to the penetration of EVs, the demand for charging facilities has also gradually surged. To this end, prosumers which can be regarded as EV charging stations (EVCSSs) [4] are focused in this article. Particularly, we concentrate on the smart meter-equipped prosumers. Nevertheless, smart meters are easy to suffer from cyber-attacks [5], despite their tamper-detection and encrypted communication capabilities [6]. Due to this defect, attackers are capable of hacking into the meters [5]. When the meters are hacked, the energy consumption readings can be tampered which can result in electricity bill reduction [5], leading to vast economic losses for the power utilities all over the world [7]. Hence, it is pivotal important to prevent cyber attacks on prosumer-based EV charging stations that are equipped with smart meters.

To prevent the cyber attacks, it is necessary to identify the cyber attacks, which can be achieved by detecting the network traffic (NT) data. However, as cyber-attacks continue to grow in sophistication [8] and the boundary between malicious and benign traffic becomes fuzzy [9], detecting cyber-attacks remains a huge challenge. Thus, a method that can identify cyber-attacks as accurately as possible needs to be designed. Centralized learning method has the potential to detect the network traffic (NT) for the prosumer-based EV charging stations. However, this method requires the centralized collection of NT data from each distributed prosumer, which may arouse data privacy issue [10]. Thanks to federated learning (FL) which is a privacy-preserving decentralized method [11], handling that issue becomes possible since FL can train a model without taking the data off-premises. Nevertheless,

Received 7 May 2024; revised 3 September 2024; accepted 8 December 2024. Date of publication 16 December 2024; date of current version 22 April 2025. This work was supported by Institute of Information & Communications Technology Planning & Evaluation (IITP) grant funded by the Korea government(MSIT) (No.2019-0-01287, Evolvable Deep Learning Model Generation Platform for Edge Computing), the National Research Foundation of Korea(NRF) grant funded by MSIT (No. RS-2023-00207816), IITP grant No.2021-0-02068, Artificial Intelligence Innovation Hub), the Convergence security core talent training business support program (IITP-2023(2023)- RS-2023-00266615), and the MSIT under the ITRC support program (IITP-2024-RS-2023-00258649) supervised by the IITP, NRF grant funded by the Korea government(MSIT) (No. RS-2024-00352423), IITP grant funded by MSIT (No. RS-2024-00509257, Global AI Frontier Lab), and KRIT grant funded by Defense Acquisition Program Administration (DAPA) (KRIT-CT-24-001). The associate editor coordinating the review of this article and approving it for publication was F. Valenza. (*Corresponding author: Choong Seon Hong.*)

Luyao Zou, Kitae Kim, Huy Q. Le, Chu Myaet Thwal, and Choong Seon Hong are with the Department of Computer Science and Engineering, Kyung Hee University, Yongin 17104, Gyeonggi, Republic of Korea (e-mail: zouluyao@khu.ac.kr; glideslope@khu.ac.kr; quanguy69@khu.ac.kr; chumyaet@khu.ac.kr; cshong@khu.ac.kr).

Quang Hieu Vo is with Samsung Electric, Suwon, South Korea (e-mail: 2019310178@khu.ac.kr).

Chaoning Zhang is with the Department of Artificial Intelligence, School of Computing, Kyung Hee University, Yongin 17104, Gyeonggi, Republic of Korea (e-mail: chaoningzhang1990@gmail.com).

Digital Object Identifier 10.1109/TNSM.2024.3517621

network traffic data of all prosumers may be non-independent and identically distributed (non-IID), which may cause FL performance degradation [12]. Besides, in practice, it is unrealistic to presume all the clients can always serve the training process [13]. Thus, there may exist a phenomenon that some clients may be reluctant to participate in the training process.

To this end, to address the aforementioned issues, an edge-assisted federated prototype knowledge distillation (E-FPKD) approach is proposed for cyber attack prevention towards prosumer-based EVCSs. To be specific, through the proposed method, firstly, cyber-attacks will be detected by using the network traffic (NT) data. Secondly, the detected cyber attacks will be intervened via rule-based method [14] at each edge server. Additionally, prior to the proposed E-FPKD approach, since feature selection process can increase the accuracy [15], feature selection with Pearson Correlation Coefficient (PCC) is adopted. For the proposed E-FPKD approach, it is designed based on the FL framework so that it permits each client can join in the training process without letting its local data leave the local side. Thus, the data privacy concern regarding submitting data to a central place can be solved. In addition, considering the non-IID NT data, we employ knowledge distillation (KD) for each client, owing to the following benefits: 1) its easiness of implementation in any off-the-shelf deep learning (DL) architecture without the need for extra software or hardware [16], 2) applying KD into FL is an effective method to deal with the data heterogeneity [17]. In addition to KD, we also employ the prototype aggregation mechanism into the considered FL framework, because of its supportability for heterogeneous clients [18]. Finally, rule-based method is triggered to intervene the detected attacks. To be friendly, the clients' willingness for joining the FL process (i.e., participating availability per client) is additionally integrated.

The cyber-attack prevention (cyber attack detection and intervention), KD, and prototype aggregation have been studied by some previous studies, respectively. For ease of understanding, brief summary of several previous studies and this article are conducted. The detailed contents will be illustrated in Section II. In regard to the cyber-attack detection for EVCS, [8], [19], [20], [21] were proposed. However, none of them consider integrating KD technique and prototype aggregation into FL framework to prevent cyber attacks on prosumer-based EV charging stations. Cyber-attack intervention was considered by [14], [22], [23]. Particularly, the preventive event was adopted by [14] for Distributed Denial-of-Service (DDoS) attacks prevention, the expert-based intervention was employed by [22], and the vulnerability assessment and penetration testing was proposed by [23] for cyber attack prevention, which differ from this work. In [24], [25], [26], [27], KD approach has been considered. Specifically, with respect to [24], [25], [26], the type of KD adopted by those three studies are different from this work. In the case of [27], although a mentor network and a mentee network are considered for building each client, prototype aggregation is not utilized. For [28], [29], [30], various KD-based FL approaches were proposed for intrusion detection, while the study [31] aims to detect the energy theft via the

method that incorporates FL with KD and model contrastive learning. However, none of them consider the prototype aggregation approach. In regard to [32], [33], [34], [35], despite all of them introducing the prototype aggregation technique, none of them simultaneously introduce feature selection, client availability reported by the client itself, KD, and prototype aggregation to detect the cyber attacks towards the prosumer-based EV charging stations. Thus, unlike [8], [14], [19], [20], [21], [22], [24], [25], [26], [27], [28], [29], [30], [31] and [32], [33], [34], [35], we comprehensively employ feature selection and meanwhile, integrate client availability and KD along with prototype aggregation in FL to detect the cyber-attacks for the prosumer-based EVCSs, while we adopt rule-based approach to intervene in the cyber-attacks. The main contributions of this work are illustrated as follows:

- We consider cyber-attacks prevention for prosumer-based EVCSs, where prosumers are regarded as the infrastructure for charging EVs. Particularly, cyber-attack detection via NT data, and cyber-attack intervention are focused.
- For cyber-attack detection, to make the boundary between benign and malicious traffic as clear as possible, maximizing the mean distance between samples from benign and malicious traffic is considered. However, this manner has a high computational cost if each prosumer has many NT data and/or has high dimensional feature space (proved in Section IV-B). Thus, instead, maximizing the overall detection correctness of all prosumers (ODC) is considered, which can achieve lower computational cost and is proven in Section IV-C. Afterward, we consider intervention against the detected cyber-attacks.
- We propose a method named edge-assisted federated prototype knowledge distillation (E-FPKD) approach which is capable of overcoming the non-IID NT data. Before utilizing the proposed E-FPKD method, the Pearson Correlation Coefficient (PCC) is adopted to select features over the NT data. Besides, we give each client the freedom to report their own availability for participating in FL process. In addition, we introduce KD for each client. Motivated by [27] which constructs each client with a mentor and a mentee model, we consider each client owns a teacher and a student network. Moreover, we introduce prototype aggregation [18] at each round, while we additionally add model aggregation in the final round to generate global model. Then a rule-based method called “if...then” [36] is employed for cyber-attacks intervention.
- Finally, a comparison experiment is conducted on three datasets, i.e., NSL-KDD [37], UNSW-NB15 [38], [39], [40], [41], [42], and IoTID20 [43]. Specifically, we compare the proposed method with several base methods, e.g., FedProto [18], pFedSD [25], FedAU [44], and FedExP [45]. Experimental results clarify that the proposed method's effectiveness is notable compared with those baselines. For instance, for IoTID20, in binary classification, the ODC achieved by the proposed method is 1.0810% and 16.10% larger than that of pFedSD and FedExP, respectively.

TABLE I
SUMMARY OF [8], [19], [20], [21] AND THIS WORK REGARDING CYBER-ATTACK DETECTION FOR EV CHARGING STATION

Ref.	Cyber-attack detection for EVCS	Prosumer-based EVCS	Remark
[8]	Yes	No	FL-based security module was proposed for cyber-attacks intrusion detection towards EVCS
[19]	Yes	No	Different machine learning techniques (e.g., decision Table classifiers) were proposed to detect the DDOS attacks for EVCS
[20]	Yes	No	IoT EVCS's malicious traffic detection via multiple machine learning-based classifier algorithms (e.g., Naive Bayes classifier)
[21]	Yes	No	DOS attacks detection for EVCS via deep learning
This work	Yes	Yes	Federated prototype knowledge distillation approach is proposed to detect cyber attacks for prosumer-based EVCS

The remainder of this article is given below. In Section II, we discuss several related works. In Section III, we show the notations of this work. In Section IV, the system model with associated problem formulation are described. Some basic knowledge regarding the proposed method is given in Section V. In Section VI, we describe the proposed E-FPKD approach. In Section VII, a comparison experiment is performed. The limitations and future potential area are provided in Section VIII. Finally, a conclusion is drawn in Section IX.

II. RELATED WORK

As aforementioned, we focus on preventing cyber attacks by detecting and intervening in the attacks on NT data. To devise the solution, we consider employing knowledge distillation in FL, while adopting the prototype aggregation to deal with the non-IID challenge. In this section, we provide the previous literature on the following aspects: 1) cyber-attack detection for EV Charging Stations (EVCSs), 2) cyber-attack intervention/prevention, 3) knowledge distillation technique in FL, and 4) prototype aggregation technique.

A. Cyber-Attack Detection for EV Charging Station

Cyber-attack detection for EV charging stations is one of the aspects considered by this article. Consequently, we list some previous studies and illustrate various points considered by this article. Currently, several studies have been proposed for the EVCS in terms of cyber-attack detection such as [8], [19], [20], [21] (summarized in Table I). Specifically, in [8], a multiple cloud-aided FL-based security module was proposed for cyber attack intrusion detection towards EVCS. Nevertheless, prosumer-based EVCS, KD and prototype aggregation were not considered by [8], whereas those points are the focus of this paper. Reference [19] aimed to detect the Distributed Denial of Service (DDoS) attack towards the EVCS network. Different from [19], we consider binary and multi-class classification scenarios. In binary classification, we do not distinguish the types of attacks; all types of attacks will be regarded as anomalies. In multi-class classification, we consider detecting various attacks. Another different point is that the FL-based approach was not taken into account by [19], whereas we consider that approach in this paper. In [20], since the risk of cyber-attacks is increasing, multiple ML-based classifier algorithms (e.g., Naive Bayes classifier,

filtered classifier) were evaluated to detect the malicious traffic of the IoT EVCS. However, prosumer-based EVCS and FL-based approach were not considered by [20]. In [21], to detect EVCS's denial of service (DOS) attacks, intrusion detection systems based on deep learning was proposed. However, prosumer-based EVCS and FL-based approach were not taken into account by [21].

B. Cyber-Attack Intervention/Prevention

In this article, we consider cyber-attack prevention, and the intervention process is one of the steps for the purpose of preventing cyber-attacks. Thus, we provide several previous studies regarding cyber-attack intervention or prevention, such as [14], [22], [23]. Moreover, we also illustrate the differences between those studies and our proposed solution. In [14], a deep learning-based method was proposed to prevent the network from being attacked by DDoS attacks and insecure data flows along with similar network intrusions, where a preventive event is triggered at the edge service, e.g., blocking data transmission. However, [14] does not employ FL, KD and prototype aggregation, which are important techniques considered by this paper. In addition, unlike [14], we consider preventing various attacks by multi-class classification in this paper. In [22], J48 decision tree algorithm was adopted to extract the patterns that are related to the cyber-attacks via historical data, and then a prediction model is built to forecast the future cyber-attacks. Afterwards, an expert will take the cyber-attacks intervention to address the cyber-attack. However, identical to [14], FL, KD and prototype aggregation techniques were not utilized by [22], either. Reference [23] provided vulnerability assessment and penetration testing for cyber attack prevention. Different from [23], we adopt an FL-based deep learning approach to detect the cyber attacks and then we employ the rule-based approach to intervene in the detected cyber-attacks.

C. Knowledge Distillation (KD) Technique in FL

In this work, KD is one of the techniques that we integrate into FL to construct the proposed solution. Thus, in this subsection, we will describe several previous literature regarding integrating KD technique in FL including [24], [25], [26], [27] (summarized in Table II). In [24], a heterogeneous FL framework was proposed, where bidirectional KD (i.e., client-to-cloud and cloud-to-client distillation) was utilized to build each client model and public dataset for server is needed.

TABLE II
SUMMARY OF [24], [25], [26], [27] AND THIS WORK REGARDING KD IN FL

Ref.	Use KD in FL	KD Type	Remark
[24]	Yes	Bidirectional KD-based FL was proposed	Public dataset is required.
[25]	Yes	Self-KD was proposed for personalized FL	Public dataset is not required.
[26]	Yes	Layer-wised KD in FL was proposed	Proxy dataset is needed.
[27]	Yes	Consider a mentor model and a mentee model to form each client.	Public dataset is not required.
This work	Yes	Same as [27], a teacher model and a client model is considered to form each client.	Public dataset is not required. Besides, feature selection stage and prototype aggregation is considered.

In particular, for client-to-cloud distillation, global network is treated as the student network, while each local network are regarded as a teacher network. Thus, the global model will unify the knowledge from each local teacher network. For cloud-to-client distillation, the global model's knowledge will be distilled for each local network. In [25], personalized FL based on self-KD was proposed, where the knowledge of previous personal model belongs to a client will be distilled to guide its current local model, and no need to use the public dataset. Different from [24] and [25], we consider each client forms by a teacher model and a student model in this article. This is inspired by [27] that adopted a mentor model and a mentee model to build each client. However, prototype aggregation was not leveraged by [27], whereas we integrate KD with prototype aggregation in each round and model aggregation in the final round. As for [26], layer-wise KD was adopted in FL, where a single teacher (server) and multiple students (multiple devices) were considered. Besides, [26] considered distilling the knowledge from various layers of the global model to teach the local student model. Moreover, [26] needs to use proxy dataset to produce knowledge. Different from [26], in this paper, proxy dataset is not required, and each client is constructed by a teacher model and a student model. Besides, we consider distilling the knowledge from the teacher model to the student model of the same client.

KD-based FL has been leveraged by several studies for various detection purposes. For instance, [28], [29], [30] have conducted research on KD-based FL approach for intrusion detection (ID), while [31] tried to detect energy theft via KD-based FL method to ameliorate energy management. Particularly, in [28], dual teacher-student paradigm based FL approach was proposed for ID, where the knowledge of two teacher models will be distilled to guide student model. This structure is different from this article, where we build each client with a teacher and a student model. In [29], a KD-based semi-supervised FL approach was presented for ID, where unlabeled data was utilized by distillation approach for enhancing the performance of classifier. Unlike [29], we use labeled data and supervised learning approach. In [30], FL ensemble KD (FLEKD) approach was proposed for ID, where the server-side model was regarded as student model, while the client models were treated as teacher models. Whereas, a client consists of a teacher and a student model is adopted by this article. In [31], federated contrastive KD was presented to detect energy theft for an urban area. However, although [31] also considered building each client by a teacher and a student

model, cyber-attack detection is not considered, whereas that is a main point of this work. Moreover, in [28], [29], [30], [31], prototype aggregation is not considered, whereas we adopt it in the proposed solution.

D. Prototype Aggregation (PA) Technique

In this part, we provide some previous studies that employ the prototype aggregation (PA), which is one of the techniques used by this work. Besides, we also point out the difference between those studies and this article. Several studies have considered using PA to address various problems, e.g., [32], [33], [34], [35]. Specifically, in [32], PA was adopted to guide the FL framework for human activity recognition. To be more specific, [32] took into account transmitting both the prototypes and the representations between server and clients. Unlike [32], in this paper, until the penultimate round, we only consider prototype transferring for the training process. In the final round, not only prototype transferring, but also model transferring are performed. In addition to this, we concentrate on cyber attack detection for prosumer-based charging stations. In [33], PA was employed in FL for predicting the heterogeneous defect. Reference [34] proposed a multi-branch prototype FL for processing the image domains in natural and medical, where PA was executed across each layer of the representation layer. Nevertheless, both [33] and [34] do not consider prosumer-based EVCSs, feature selection and KD. PA was also adopted by [35], where prototype per device will be transmitted to the neighbors of that device for aggregation. Differ from this, we consider sending local prototypes to distribution system operator (DSO) for aggregation.

III. NOMENCLATURE

In this section, we illustrate the related notations that are used in this work, which are shown in Table III.

IV. SYSTEM MODEL AND PROBLEM FORMULATION

Considering the prosumer-based EV charging stations (EVCSs) that are composed of a set $\mathcal{P} = \{1, 2, \dots, P\}$ of prosumers, a set $\mathcal{S} = \{1, 2, \dots, S\}$ of dedicated local edge servers (DLESSs), a set of charging service required EVs, a distribution system operator (DSO) and the power grid, shown as Fig. 1. In particular, we consider each prosumer has a smart meter (SM) that is equipped with tamper-detection and encrypted communication function [6]. Besides, each prosumer is embedded with a solar panel (SP) along with at

TABLE III
SUMMARY OF NOTATIONS

Notation	Definition	Notation	Definition
Section IV			
$\mathcal{P}$	Set of prosumers	$\mathcal{S}$	Set of dedicated local edge servers (DLES)
$\mathcal{T}_p$	Set of time slots for prosumer p	$\mathcal{Y}_s$	Labels for Network traffic (NT) data of prosumer p
$\mathcal{X}_s$	NT data of prosumer p that connects with DLES s	$\hat{y}_s^t$	Detected result for NT data at time slot t
$\Omega_s(\cdot)$	Representation layer of deep learning (DL) model in the DLES s	$\Lambda_s(\cdot)$	Classifier of DL model in the DLES s
φ_s^t	Features extracted from the representation layer	ξ_s^{boundary}	Boundary between samples from various categories for each prosumer p
$\tilde{\mathcal{X}}_s$	Set of malicious traffic for prosumer p	$\tilde{\mathcal{X}}_s$	Set of benign traffic for prosumer p
$DIST(\cdot, \cdot)$	Distance between output of the representation layer for a malicious traffic and for a benign traffic	$ \tilde{\mathcal{X}}_s \text{ \& } \tilde{\mathcal{X}}_s $	Number of malicious samples and benign samples, respectively, for prosumer p
Δ_p	Total number of operations of eq. (2)	$\mathcal{E}_{dist}$	Number of mathematical operations for calculating $DIST(\varphi_s^t, \varphi_s^t)$
$\mathbb{1}_{y_s^t=0}$	Indicator function (if the detected result belongs to malicious traffic, then this value will be 1)	$\mathbb{1}_{y_s^t=1}$	Indicator function (if the detected result is normal, then this value will be 1)
Section V			
ω_m	Local model of client m	ω	Global model
$\mathcal{D}_m$	Dataset owned by client m	$\mathcal{D}$	Overall dataset
$ \mathcal{D}_m $	Data size of each client m	$ \mathcal{D} $	Data size of all clients
$\mathcal{L}^{KD} \text{ \& } \zeta$	KD loss and Temperature, respectively	$KL(\cdot, \cdot)$	KL divergence function
$\varrho^k(x)$	Soft probability of Softmax_T (TNet)	$\hat{\varrho}^k(x)$	Soft probability of Softmax_T (SNet)
$b(x)$	A vector of scores generated by the TNet	$\bar{N}^{(k)}$	Global prototype of each class k
$N_m^{(k)}$	Prototype of the class k owned by client m	$\mathcal{M}_k$	Set of the clients that own class k
$\mathcal{D}_{m,k}$	Subset of local dataset $\mathcal{D}_m$ (belong to k^{th} category)	$ \mathcal{D}_{m,k} $	Overall data size of $\mathcal{D}_{m,k}$
Section VI			
$\mathcal{T}$	All time slots	x^t	Network traffic data at time slot t
$\mathcal{X}$	Network traffic data of all time slots	$i \text{ and } j$	Two features of $\mathcal{X}$
x_i	Vector of feature i	x_j	vector of feature j
λ_{ij}	PCC between feature i and feature j	$\bar{x}_i$	Average value of all the elements in the vector x_i
$\bar{x}_j$	Average value of all the elements in the vector x_j	α_s	Availability per client
$b_s(x_s^t)$	Output of the teacher network per client (DLES s)	x_s^t	Network traffic data at time slot t of DLES s
$\varrho_s^k(x_s^t)/\varrho_s^k(x_s^t)$	Probability for TNet/SNet by using Softmax_T	K	Total number of classes
$b_s^u(x_s^t)$	u -th element of the vector $b_s(x_s^t)$	$\mathcal{L}_{s,SN}^{KD}$	Kullback-Leibler (KL) divergence of client s
N_s	Prototype set of each participating client s	$N_s^{(k)}$	Prototype of class k for the participating client s
$\Omega_s(\theta_s; x_s^t)$	Embedding function for the representation layer of client s with the parameter θ_s	$\mathcal{D}_{s,k}$	Training NT data and labels of client s that belong to the k^{th} category
$\mathcal{D}_s$	Local network traffic dataset	$\mathcal{L}_{s,SN}^R$	Regularization term
$distance(\cdot, \cdot)$	Distance metric	$\mathcal{L}_{s,SN}$	Local loss function
$\mathcal{L}_{s,SN}^{SL}$	Supervised learning loss per student network	$\mathcal{L}_{s,SN}^{\text{entropy}}$	BCE loss or CE loss
$\psi \text{ \& } \gamma$	Hyperparameters	ϑ_s	Parameter of classifier of client s
$ D_{s,k} $	Data size of $D_{s,k}$	$\hat{H}$	Total number of the network traffic data owned by the participating clients
$Q \text{ \& } \hat{\eta}$	Final round and learning rate, respectively	ω_s	Local model for SNet of client s
$ \mathcal{X}_s $	Total number of NT data owned by client s	U	Overall number of all participating clients' instances
U_k	Instances number of class k over all participating clients	G^{nf}	Number of filters in CNN
G^{ks}	Kernel size	G^{ne}	Number of elements of the input vector
G^{ts}	Number of time steps	e_t	$(e_t)^{th}$ convolutional layer of teacher network
$\hat{e}_t$	$\hat{e}_t^{th}$ fully connected (FC) layer	$G_{e_t-1}^{nu}$	No. of the neural units (NUs) in the FC layer $\hat{e}_t - 1$
$G_{\hat{e}_t}^{nu}$	No. of the NUs in the FC layer $\hat{e}_t$	$\hat{E}$	Local iteration
Z	Time complexity of the backbone network for SNet	K_s	No. of prototypes generated by each SNet
e_s	$(e_s)^{th}$ convolutional layer of SNet	$\hat{e}_s$	$(\hat{e}_s)^{th}$ FC layer of SNet
$G_{e_s-1}^{nu}$	NUs in the FC layer $\hat{e}_s - 1$	$G_{\hat{e}_s}^{nu}$	NUs in the layer $\hat{e}_s$ of FC network
M	No. of participating clients in each round		
Section VII			
δ	Parameter for Dirichlet Distribution	TP	Correctly detected as attacks
TN	Correctly detected as normal traffic	FP	Normal traffic is incorrectly detected as attack
FN	Attack traffic is incorrectly detected as the normal traffic	Υ_{Acc}	Accuracy
Υ_{Acc}	Average accuracy (AA)	Υ_{Acc}^s	Accuracy of client s
Υ_{Prec}	Precision	Υ_{Prec}^s	Average precision (AP)
Υ_{Prec}^s	Precision of client s	Υ_{Rcl}	Recall
Υ_{Rcl}	Average recall (AR)	Υ_{Rcl}^s	Recall of each client s
Υ_{F1_score}	F1 score	$\Upsilon_{F1_score}^s$	Average F1 score
$\Upsilon_{F1_score}^s$	F1 score of client s	Υ_{FAR}	False alarm rate (FAR)
Ξ_{cdr}	No. of the correct detected results	$\Xi_{overall}$	Overall number of network traffic data in a dataset

least an EV supply equipment (EVSE), which is regarded as the physical infrastructure for supplying energy to public EVs. Moreover, each prosumer is considered connecting with the DSO for purchasing power grid energy. Further, we assume each prosumer $p \in \mathcal{P}$ is associated with a DLES $s \in \mathcal{S}$ [4].

A. Cyber-Attack Detection Model via Deep Learning

For the considered prosumer-based EVCS, in order to prevent cyber attacks, it is necessary to correctly detect cyber attacks. Thus, cyber-attack detection model (CDM) is investigated. Particularly, we focus on training a deep learning

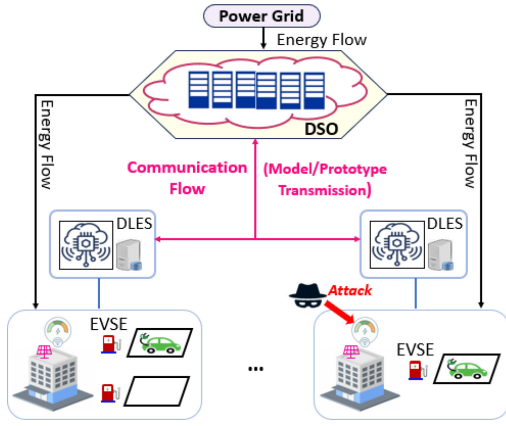


Fig. 1. System model of the prosumer-based EV charging stations.

(DL)-based CDM by using the network traffic (NT) data of the considered EVCSs, which can distinguish the malicious traffic from the benign traffic. Generally, the DL model is composed of 1) feature extraction layers (i.e., representation layers [18]), 2) classification decision-making layers [18] (a.k.a. classifier).

For each prosumer $p \in \mathcal{P}$, we assume its connected DLES owns its NT data. Thus, we deploy the DL model in each connected DLES s . In addition, we consider a set $\mathcal{T}_p = \{1, 2, \dots, T_p\}$ of T_p time slots. As a result, the NT data of prosumer p at all time slots can be denoted as $\mathcal{X}_s = \{x_s^1, x_s^2, \dots, x_s^t, x_s^t \in \mathbb{R}^d, t \in \mathcal{T}_p\}$ and the corresponding labels can be represented as $\mathcal{Y}_s = \{y_s^1, y_s^2, \dots, y_s^t\}$. It is worthwhile to mention that $\mathbb{R}^d$ represents the d -dimensional Euclidean space [46]. Let $\Omega_s(\cdot)$ and $\Lambda_s(\cdot)$ denote the representation layers and the classifier of DL model in the DLES s , respectively. Hence, we can define the extracted features at time slot t , φ_s^t , as $\varphi_s^t = \Omega_s(x_s^t)$, which is leveraged as the input of the classifier. For the detected result $\hat{y}_s^t$, as per [9], it can be obtained by,

$$\hat{y}_s^t = (\Lambda_s \circ \Omega_s)(x_s^t) = \Lambda_s(\Omega_s(x_s^t)) = \Lambda_s(\varphi_s^t), \quad (1)$$

where $\circ$ indicates the composition of functions in mathematics.

B. Malicious and Benign Traffic Boundary Model

To detect the cyber-attacks, it is possible to be achieved by distinguishing the malicious traffic (i.e., anomaly) from the benign traffic (i.e., normal). Due to the boundary between those two categories (i.e., inter-class distance [9]) becoming ambiguous, many false detection can occur [9]. Thus, it is still necessary to be dedicated to distinguish the boundary between those two categories (i.e., malicious category and benign category) as clearly as possible.

To facilitate the definition of the boundary between benign and malicious traffic per prosumer p , we define the malicious traffic set as $\hat{\mathcal{X}}_s = \{\hat{x}_s^1, \hat{x}_s^2, \dots, \hat{x}_s^{t_1}\}$ and the benign traffic set as $\tilde{\mathcal{X}}_s = \{\tilde{x}_s^1, \tilde{x}_s^2, \dots, \tilde{x}_s^{t_2}\}$, where $\mathcal{X}_s = \hat{\mathcal{X}}_s \cup \tilde{\mathcal{X}}_s$. According to [9], the boundary between two categories can be represented as the mean distance between samples from those two categories (i.e., inter-class distance [9]). Hence, assuming

$\hat{\mathcal{X}}_s$ and $\tilde{\mathcal{X}}_s$ are not empty, the mentioned boundary can be given as follows:

$$\xi_s^{\text{boundary}} = \frac{1}{|\hat{\mathcal{X}}_s| |\tilde{\mathcal{X}}_s|} \sum_{t_1=1}^{|\hat{\mathcal{X}}_s|} \sum_{t_2=1}^{|\tilde{\mathcal{X}}_s|} \text{DIST}(\varphi_s^{t_1}, \varphi_s^{t_2}), \quad (2)$$

where $\text{DIST}(\varphi_s^{t_1}, \varphi_s^{t_2})$ represents the distance between $\varphi_s^{t_1}$ and $\varphi_s^{t_2}$. Here, $\varphi_s^{t_1}$ and $\varphi_s^{t_2}$ can be obtained by representation layers. As for $|\hat{\mathcal{X}}_s|$ and $|\tilde{\mathcal{X}}_s|$, they are used to indicate the number of malicious samples and benign samples, respectively.

For the boundary distinction between malicious category and benign category, as per [9], it can be realized by maximizing the inter-class distance (maximizing ξ_s^{boundary}). However, this method may cost high computational cost, which can be known through the following *Lemma* and its *proof*.

Lemma 1: The computational cost of eq. (2) is high, if there exists many network traffic data per prosumer and/or high-dimensional feature space.

Proof: In eq. (2), for the $\text{DIST}(\varphi_s^{t_1}, \varphi_s^{t_2})$, various methods (L1-distance, L2-distance, etc.) can be chosen for calculation. Consequently, the number of mathematical operations relies on the selected method for calculating the distance. For convenience, suppose $\mathcal{E}_{\text{dist}}$ denotes the number of mathematical operations for calculating $\text{DIST}(\varphi_s^{t_1}, \varphi_s^{t_2})$. Thus, according to eq. (2), $\text{DIST}(\varphi_s^{t_1}, \varphi_s^{t_2})$ needs to be calculated $|\hat{\mathcal{X}}_s| |\tilde{\mathcal{X}}_s|$ times. Besides, we also need to compute a multiplication and a division operation for obtaining $\frac{1}{|\hat{\mathcal{X}}_s| |\tilde{\mathcal{X}}_s|}$. Namely, two additional calculations are needed. Accordingly, the total number of operations of eq. (2) is $\Delta_p = |\hat{\mathcal{X}}_s| |\tilde{\mathcal{X}}_s| \mathcal{E}_{\text{dist}} + 2$. As a result, if $|\hat{\mathcal{X}}_s|$, $|\tilde{\mathcal{X}}_s|$ and/or $\mathcal{E}_{\text{dist}}$ are very large, the computational cost for eq. (2) will be high. ■

C. Problem Formulation

In this work, we concentrate on the cyber-attack prevention issue for the considered prosumer-based EVCSs. Specifically, we consider detecting cyber attacks in NT data first, then performing a cyber attack intervention process. The contents regarding intervention is explained in Section VI. Thus, next we will only explain the purpose for cyber-attack detection.

According to *Lemma 1* with its *proof*, when each prosumer owns a large amount of NT data and/or high-dimensional feature space, using eq. (2) for malicious and benign traffic data distinguishment will encounter the computational cost issue. Therefore, reducing the computational cost of distinguishing the malicious and benign traffic needs to be tackled.

To this end, in order to mitigate the computational cost, instead of maximizing eq. (2), we consider maximizing the overall detection correctness of all prosumers (ODC). It is worth noting that maximizing the ODC has less computational cost compared with maximizing the eq. (2) for all prosumers. To prove this point, we need to define ODC first. To define ODC clearly, two indicator functions, $\mathbb{1}_{y_s^t=0}$ and $\mathbb{1}_{y_s^t=1}$, are introduced, which are separately given as follows:

$$\mathbb{1}_{y_s^t=0} = \begin{cases} 1, & \text{if } \hat{y}_s^t = 0, \\ 0, & \text{otherwise.} \end{cases} \quad (3)$$

$$\mathbb{1}_{y_s^t=1} = \begin{cases} 1, & \text{if } \hat{y}_s^t = 1, \\ 0, & \text{otherwise.} \end{cases} \quad (4)$$

Here, $y_s^t = 0$ means the network traffic data at time slot t actually is malicious traffic. If the detected result is also malicious (i.e., $\hat{y}_s^t = 0$), then $\mathbb{1}_{y_s^t=0}$ is equal to 1. Otherwise, $\mathbb{1}_{y_s^t=0} = 0$, which means the detected result is wrong. For $y_s^t = 1$, it represents the network traffic data at time slot t is benign traffic in fact. At this situation, if the detected result is correct (the detected result is also benign ($\hat{y}_s^t = 1$)), then $\mathbb{1}_{y_s^t=1} = 1$. Otherwise, $\mathbb{1}_{y_s^t=1} = 0$, which indicates the detected result is malicious traffic.

Accordingly, the ODC maximization problem can be defined as follows:

$$\max \sum_{s=1}^{|S|} \sum_{t=1}^{|\mathcal{T}_p|} \left(\mathbb{1}_{y_s^t=0} + \mathbb{1}_{y_s^t=1} \right). \quad (5)$$

It is noteworthy that since we consider a prosumer p connects with a DLES, we have $|S| = |\mathcal{P}|$. Here, $|\mathcal{P}|$ means the total number of prosumers in the considered EV charging stations. As for the computational cost of (5), since a traffic data is either normal or abnormal, the calculation cost of all the prosumers by (5) is $|S||\mathcal{T}_p| = |S|(|\hat{\mathcal{X}}_s| + |\tilde{\mathcal{X}}_s|)$. Compared the computational cost of (2) and eq. (5) for all prosumers, we can get the following *Lemma* with its proof.

Lemma 2: The computational cost of (5) is less than maximizing eq. (2) for all prosumers under some considerations.

Proof: For eq. (2), considering applying it for all prosumers, then the computational cost can be obtained as $|S|\Delta_p = |S|(|\hat{\mathcal{X}}_s||\tilde{\mathcal{X}}_s|\mathcal{E}_{dist} + 2)$. According to the definition of $|\hat{\mathcal{X}}_s|$, $|\mathcal{X}_s|$ and $\mathcal{E}_{dist}$, since we consider $\hat{\mathcal{X}}_s \neq \emptyset$ and $\tilde{\mathcal{X}}_s \neq \emptyset$, it can be known that $|\hat{\mathcal{X}}_s| \geq 1$, $|\tilde{\mathcal{X}}_s| \geq 1$ and $\mathcal{E}_{dist} \geq 1$. Next, we will illustrate the following cases to prove *Lemma 2*.

- *Case 1:* If $|\hat{\mathcal{X}}_s| = 1$, $|\tilde{\mathcal{X}}_s| = 1$ and $\mathcal{E}_{dist} \geq 1$, then the computational cost of applying eq. (2) can be calculated as $|S|(|\hat{\mathcal{X}}_s||\tilde{\mathcal{X}}_s|\mathcal{E}_{dist} + 2) = |S|(\mathcal{E}_{dist} + 2)$. Because $\mathcal{E}_{dist} \geq 1$, $|S|(\mathcal{E}_{dist} + 2) \geq 3|S|$. In the case of the computational cost of (5), it will be $|S|(|\hat{\mathcal{X}}_s| + |\tilde{\mathcal{X}}_s|) = 2|S|$. Hence, for this case, *Lemma 2* can be proved to be true.
- *Case 2:* If $|\hat{\mathcal{X}}_s| = 1$, $|\tilde{\mathcal{X}}_s| > 1$ and $\mathcal{E}_{dist} \geq 1$, then we can obtain $|S|(|\hat{\mathcal{X}}_s||\tilde{\mathcal{X}}_s|\mathcal{E}_{dist} + 2) = |S|(|\mathcal{X}_s|\mathcal{E}_{dist} + 2) \geq |S|(|\tilde{\mathcal{X}}_s| + 2)$. The computational cost of (5) can be computed as $|S|(|\hat{\mathcal{X}}_s| + |\tilde{\mathcal{X}}_s|) = |S|(|\tilde{\mathcal{X}}_s| + 1)$. Thus, in this case, we can know $|S|(|\hat{\mathcal{X}}_s||\tilde{\mathcal{X}}_s|\mathcal{E}_{dist} + 2) > |S|(|\hat{\mathcal{X}}_s| + |\tilde{\mathcal{X}}_s|)$. Accordingly, for this case, *Lemma 2* is true.
- *Case 3:* If $|\hat{\mathcal{X}}_s| > 1$, $|\tilde{\mathcal{X}}_s| = 1$ and $\mathcal{E}_{dist} \geq 1$, then $|S|(|\hat{\mathcal{X}}_s||\tilde{\mathcal{X}}_s|\mathcal{E}_{dist} + 2) = |S|(|\hat{\mathcal{X}}_s|\mathcal{E}_{dist} + 2) \geq |S|(|\hat{\mathcal{X}}_s| + 2)$, and $|S|(|\hat{\mathcal{X}}_s| + |\tilde{\mathcal{X}}_s|) = |S|(|\hat{\mathcal{X}}_s| + 1)$. Thus, $|S|(|\hat{\mathcal{X}}_s||\tilde{\mathcal{X}}_s|\mathcal{E}_{dist} + 2) > |S|(|\hat{\mathcal{X}}_s| + |\tilde{\mathcal{X}}_s|)$, thereby, *Lemma 2* is correct for this case.
- *Case 4:* If $|\hat{\mathcal{X}}_s| \geq |\tilde{\mathcal{X}}_s| > 1$, and $\mathcal{E}_{dist} \geq 1$, then $|S|(|\hat{\mathcal{X}}_s||\tilde{\mathcal{X}}_s|\mathcal{E}_{dist} + 2) > |S||\tilde{\mathcal{X}}_s||\tilde{\mathcal{X}}_s|\mathcal{E}_{dist}$. In addition, in this article, $|\mathcal{X}_s|$ is adopted to represent the number of the samples that belong to the benign traffic. Thus, it is integer. Accordingly, since $|\tilde{\mathcal{X}}_s| > 1$, it can be inferred that $|\tilde{\mathcal{X}}_s| \geq 2$. Therefore, it can be obtained that $|S||\tilde{\mathcal{X}}_s||\tilde{\mathcal{X}}_s|\mathcal{E}_{dist} \geq 2|S||\tilde{\mathcal{X}}_s|$. For the computational cost of (5), because $|\mathcal{X}_s| \geq |\tilde{\mathcal{X}}_s|$, $|S|(|\hat{\mathcal{X}}_s| + |\tilde{\mathcal{X}}_s|) \leq |S|(|\hat{\mathcal{X}}_s| +$

$|\hat{\mathcal{X}}_s|) = 2|S||\hat{\mathcal{X}}_s|$. Accordingly, we can obtain that $|S|(|\hat{\mathcal{X}}_s||\tilde{\mathcal{X}}_s|\mathcal{E}_{dist} + 2) > |S|(|\hat{\mathcal{X}}_s| + |\tilde{\mathcal{X}}_s|)$. Consequently, for this case, *Lemma 2* is also true.

- *Case 5:* If $|\tilde{\mathcal{X}}_s| \geq |\hat{\mathcal{X}}_s| > 1$, and $\mathcal{E}_{dist} \geq 1$, since $|\hat{\mathcal{X}}_s|$ denotes the number of malicious traffic samples, we can infer $|\hat{\mathcal{X}}_s| \geq 2$. Hence, similar as *Case 4*, we can prove that $|S|(|\hat{\mathcal{X}}_s||\tilde{\mathcal{X}}_s|\mathcal{E}_{dist} + 2) > |S||\hat{\mathcal{X}}_s||\tilde{\mathcal{X}}_s|\mathcal{E}_{dist} \geq 2|S||\hat{\mathcal{X}}_s|$ and $|S|(|\hat{\mathcal{X}}_s| + |\tilde{\mathcal{X}}_s|) \leq |S|(|\hat{\mathcal{X}}_s| + |\tilde{\mathcal{X}}_s|) = 2|S||\tilde{\mathcal{X}}_s|$. Thus, we can prove *Lemma 2* is correct in this case.

According to the analysis from Case 1 to Case 5, we can prove under some considerations, *Lemma 2* is correct. ■

V. PRELIMINARIES KNOWLEDGE

To prevent the considered prosumer-based EVCSs from suffering cyber-attacks, we propose an edge-assisted federated prototype knowledge distillation (E-FPKD) approach. Before explaining the proposed E-FPKD, as federated learning, knowledge distillation, and prototype aggregation are used to design the proposed solution, for ease of understanding, it is worth to explain those three techniques first before going into the depth of the proposed solution in Section VI.

A. Background: Federated Learning (FL)

The proposed E-FPKD is a FL-based approach. To understand the structure of the proposed solution more easily, we will illustrate the background knowledge of FL in this part.

In edge computing, an emerging approach named FL is widely used, which is composed of a server and several clients [47]. FL aims to collaboratively train machine learning models among various clients [25] without the necessity of letting the data leave their local side. When it comes to FL, it is worth mentioning that a commonly utilized approach is termed FederatedAveraging (i.e., FedAvg) proposed by [48]. As shown in Fig. 2, the procedure of FedAvg consists of 4 steps, which will be iterated as per [48]. Those 4 steps are shown as below:

- *Step 1:* The global model will be distributed to each local client to initialize each local model.
- *Step 2:* Each local client m will perform local training to update local model ω_m .
- *Step 3:* The updated local model parameters will be transmitted to the server [49].
- *Step 4:* The model aggregation will be performed on the server to generate new global model. Particularly, the global model ω can be obtained as follows [50]:

$$\omega = \sum_m \frac{|\mathcal{D}_m|}{|\mathcal{D}|} \omega_m, \quad (6)$$

where $\mathcal{D} = \sum_m \mathcal{D}_m$ indicates the overall dataset and $\mathcal{D}_m$ denotes the dataset of client m . $|\mathcal{D}_m|$ and $|\mathcal{D}|$ indicate the data size of each client and all the clients, respectively.

Example 1: Consider a FL-based network intrusion detection system (IDS) across various organizations, for this IDS, we consider the model aggregation is executed in the server to generate the global model, and each organization is considered as each client of the FL process and owns a local model.

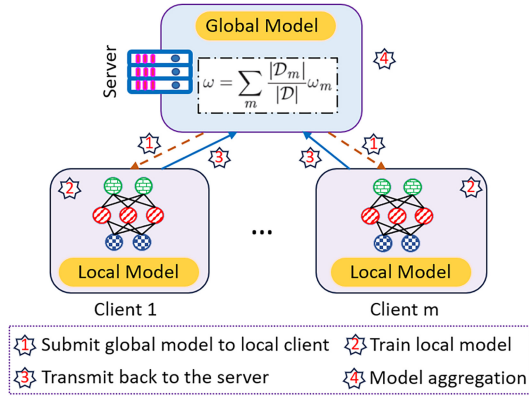


Fig. 2. Training process of federated learning approach.

In the beginning, the server will send the global model to each organization to initialize each local model. Then each organization will train the local model via its local network traffic (NT) data such that to update the local model. Next, the updated local model will be transmitted to the server for model aggregation to generate a new global model.

B. Background: Knowledge Distillation (KD)

Considering the NT data of each prosumer may be heterogeneous, since integrating KD into FL is an efficient way for handling the data heterogeneity issue [17], KD technique is introduced to be a composition of the proposed approach. To make it easier to understand the proposed method, we provide some background knowledge of KD in this part.

KD is the approach that can benefit the training process of the smaller student network (SNet) from the supervision of a larger teacher network (TNet) [51]. Specifically, the knowledge of TNet will be distilled to SNet. To do this, *Kullback-Leibler (KL) divergence* can be employed to enable the SNet to imitate the behavior of the TNet [52]. Thus, the KD loss, $\mathcal{L}^{KD}$, can be given as below [52]:

$$\mathcal{L}^{KD} = \frac{1}{|D_m|} \sum_{t=1}^{|D_m|} \zeta^2 KL(\varrho^k(x), \widehat{\varrho}^k(x)), \quad (7)$$

where $KL(\cdot, \cdot)$ is the KL divergence function, while ζ denotes the relaxation hyperparameter (i.e., temperature) [52]. In (7), x is the input data sample. $\varrho^k(x)$ and $\widehat{\varrho}^k(x)$ are the soft probability of TNet and SNet by using *softmax with temperature scaling* [53], respectively.

Example 2: Consider designing a network intrusion detection system for a mobile device. Suppose there is a pre-trained model that can detect malicious traffic well, however, that model is too large to be deployed on a resource-limited mobile device. Thus, KD is used to create a smaller model. To be specific, assume there is a lightweight model that is deployable to the mobile device. When training that lightweight model, we distill the knowledge from the pre-trained model to the lightweight model by minimizing the KD loss (i.e., eq. (7)).

C. Background: Prototype Aggregation (PA)

Prototype aggregation (PA) mechanism is another technique that we will integrate into FL to construct the proposed method (Section VI). Thus, for ease of understandability towards the proposed solution, it is valuable to illustrate PA beforehand.

Prototype is defined as the average value of the embedding vectors for the instances in class k [18], where the embedding vectors are the extracted features from the representation layers. Notably, according to [18], one prototype is used to indicate one class. Similar to the original FedAvg that adopts model aggregation, for **prototype aggregation**, the prototypes of each category received from all participated clients will be aggregated to generate new global prototype of each class k , $\overline{N}^{(k)}$, which can be given as follows [18]:

$$\overline{N}^{(k)} = \frac{1}{|\mathcal{M}_k|} \sum_{m \in \mathcal{M}_k} \frac{|D_{m,k}|}{\sum_{m \in \mathcal{M}_k} |D_{m,k}|} N_m^{(k)}, \quad (8)$$

where $N_m^{(k)}$ denotes the prototype of class k owned by client m , $\mathcal{M}_k$ represents a set of the clients who participated in the FL process that own class k , and $|D_{m,k}|$ indicates the number of the data of client m that belongs to class k . More details will be provided in the next Section.

VI. OVERALL ARCHITECTURE DESIGN FOR THE PROPOSED METHODOLOGY

Based on the background knowledge illustrated in the previous section, we design the method for cyber-attack prevention towards the considered prosumer-based EVCSs.

Fig. 3 shows the structure of the proposed solution, which will be illustrated next. Firstly, feature selection (FS) process for network traffic (NT) data preprocessing is employed since FS is beneficial for increasing accuracy [15]. Secondly, considering the privacy issue per prosumer and the heterogeneous data distribution among each prosumer, an edge-assisted federated prototype knowledge distillation (E-FPKD) approach is proposed for cyber-attacks prevention, where this method is designed based on the incorporation of federated learning, knowledge distillation (KD), prototype aggregation and rule-based method [14]. Particularly, we deploy the DL-based CDM on each DLES (dedicated local edge server) and treat each DLES as an E-FPKD client. To be more specific, the proposed E-FPKD approach contains two parts: cyber-attacks detection by solving eq. (5), and cyber-attacks intervention. To detect cyber-attacks, we combine KD and prototype aggregation with the FL approach. Besides, in practice, it is unrealistic to suppose all clients are always able to offer the training service [13], which may cause some clients to be unwilling to join the training process. Thus, each client's availability is adopted in this article. Accordingly, as shown in Fig. 3, we introduce the local client by the following order: 1) the availability of each client for joining the FL process, 2) how to use KD to construct the local client (teacher network and student network), 3) the prototypes gained by each local SNet, and 4) the objective of each local SNet. As for the DSO cloud, availability-based aggregation process for cyber-attacks detection is conducted, which contains 1) prototype

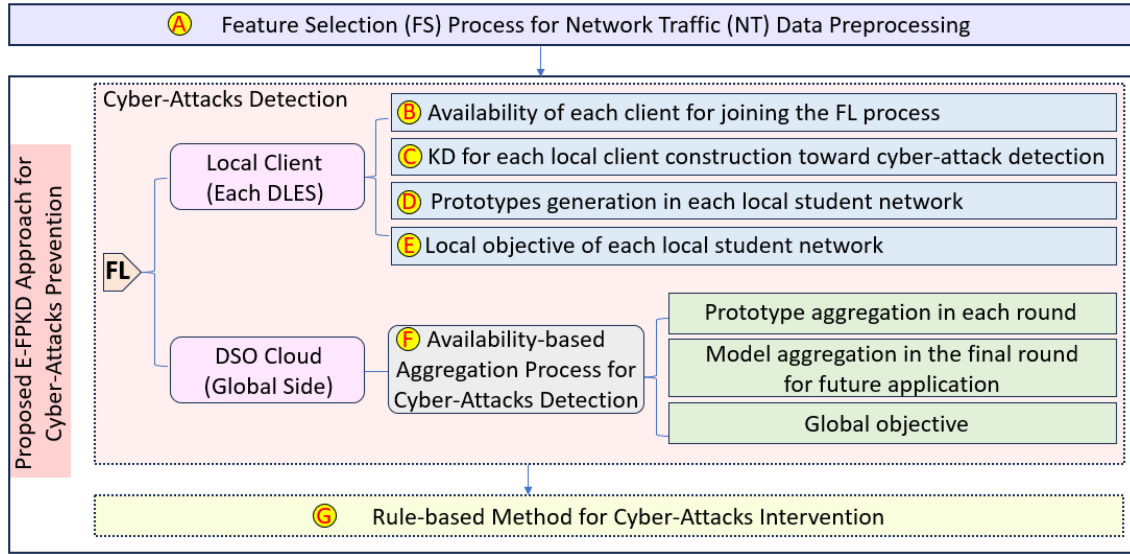


Fig. 3. Structure of the proposed solution for cyber-attack prevention towards the considered prosumer-based EVCSs.

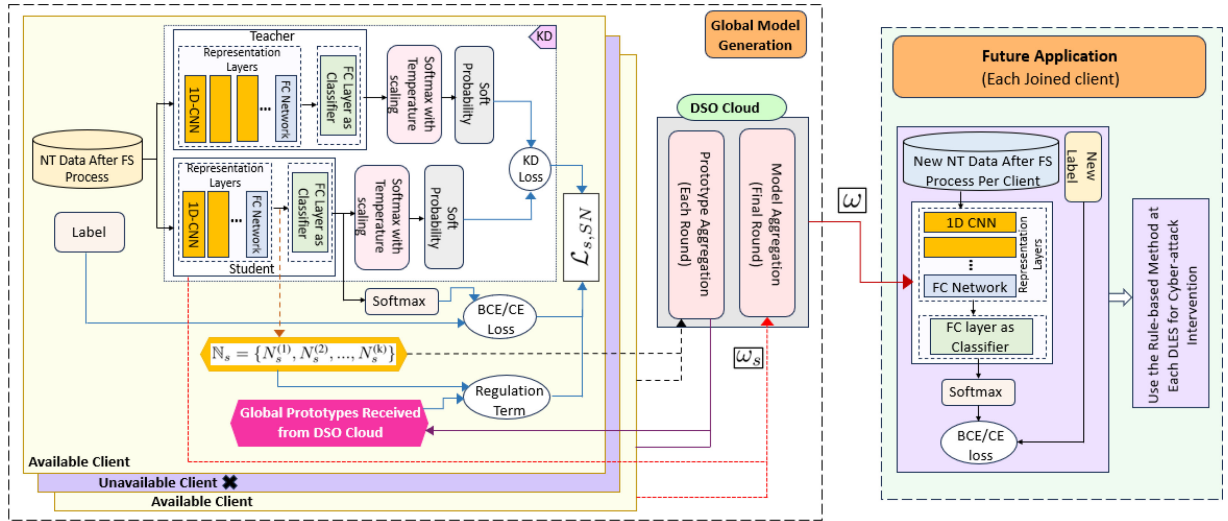


Fig. 4. Overall architecture of the proposed E-FPKD approach towards cyber-attacks prevention in both detection and intervention.

aggregation in each round, 2) model aggregation in the final round for future application, and 3) the global objective. After detection, the detected cyber attacks will be intervened at each edge via the rule-based method. It is noteworthy that the proposed solution is advantageous in handling NT data with non-IID nature and can protect privacy to a certain extent. The reasons are as follows:

- The proposed E-FPKD approach is constructed based on the FL. For FL, it can ensure data privacy by saving data and training the model on each client locally [54]. Thus, the proposed E-FPKD approach can also assure the data privacy issue caused by central data collection.
- However, FL faces the challenge of handling non-IID data [54]. Since applying KD in FL can help deal with the data heterogeneity issue [17], we introduce KD into FL to build the proposed E-FPKD approach. Motivated by [27] that considers each client to consist of a mentee model and a mentor model, we employ a teacher model

and a student model to form each client. Hence, each client will have its own teacher model, which can better adapt to the local datasets' characteristics.

- Apart from KD, prototype aggregation proposed by [18] is also employed to create the proposed E-FPKD approach. Prototype aggregation has the following goodness given by [18]: 1) through prototype aggregation, the identical label space can enable the embedding space of the participating client to be shared, 2) the information is capable of exchanging efficiently across all the participating heterogeneous clients, and 3) the heterogeneous class spaces can be supported. Hence, integrating prototype aggregation can further help process non-IID data.

Sections VI-A ~ VI-G are organized by (A) ~ (G) shown in Fig. 3, while Section VI-H summarizes the proposed E-FPKD algorithm towards cyber-attack prevention. Fig. 4 shows the overall architecture of the proposed E-FPKD approach. The detailed contents will be described next.

Algorithm 1 Feature Selection With Pearson Correlation Coefficient

Input: Network traffic (NT) data:

$$\mathcal{X} = \{x^1, x^2, \dots, x^t\}$$

Output: $\hat{\mathcal{X}}$

- 1: Get NT data vector for each feature based on $\mathcal{X}$
- 2: Calculate PCC for each two features via (9)
- 3: Check whether $0.1 \leq \text{PCC} \leq 1$ or $-1 \leq \text{PCC} \leq -0.1$
- 4: Retain features according to the introduced heuristic rule
- 5: Get NT data after feature selection process: $\hat{\mathcal{X}}$
- 6: **return** NT data after feature selection: $\hat{\mathcal{X}}$

A. Feature Selection for NT Data Preprocessing

Feature selection is a critical step for optimizing learning complexity through prioritizing the features, especially for the network traffic data [55]. Through the feature selection process, the irrelevant features can be excluded [15].

In this article, Pearson Correlation Coefficient (PCC) is employed for feature selection. Particularly, PCC was introduced in 1895 by Pearson [56], which adopts bivariate analysis [15] that is capable of evaluating the strength of the relationship between two vectors [57]. Let $\mathcal{T} = \{1, 2, \dots, T\}$ denote all time slots of network traffic (NT) data. As described in Section IV-A, we consider the NT data of each time slot t belongs to $\mathbb{R}^d$ (d -dimensional Euclidean space), without loss of generality, we denote the NT data at time slot t as $x^t = \{x_1^t, x_2^t, \dots, x_d^t\}$, $t \in \mathcal{T}$. Then the NT data of all time slots can be denoted as $\mathcal{X} = \{x^1, x^2, \dots, x^t\}$. For two features of $\mathcal{X}$, i and j , the vector of feature i can be represented as $x_i = \{x_i^1, x_i^2, \dots, x_i^t\}$, while the vector of feature j can be denoted as $x_j = \{x_j^1, x_j^2, \dots, x_j^t\}$, $t \in \mathcal{T}$. Then we can calculate the PCC between feature i and feature j , λ_{ij} , as follows [15]:

$$\lambda_{ij} = \frac{\sum_{t=1}^{|\mathcal{T}|} (x_i^t - \bar{x}_i) (x_j^t - \bar{x}_j)}{\sqrt{\sum_{t=1}^{|\mathcal{T}|} (x_i^t - \bar{x}_i)^2} \sqrt{\sum_{t=1}^{|\mathcal{T}|} (x_j^t - \bar{x}_j)^2}}. \quad (9)$$

Here, $\bar{x}_i = \frac{\sum_{t=1}^{|\mathcal{T}|} x_i^t}{|\mathcal{T}|}$ and $\bar{x}_j = \frac{\sum_{t=1}^{|\mathcal{T}|} x_j^t}{|\mathcal{T}|}$ [58]. As per [56], the range of PCC value is $[-1, +1]$. In particular, PCC in the range $\pm 0.1 - \pm 0.3$ means weak relation, in the range $\pm 0.3 - \pm 0.5$ represents moderate relation, in the range $\pm 0.5 - \pm 1.0$ indicates strong relation, and if PCC is equal to ± 1 means perfect relation [59]. As a result, we consider $0.1 \leq \text{PCC} \leq 1$ or $-1 \leq \text{PCC} \leq -0.1$ as the rule for choosing features.

Algorithm 1 lists the process of the feature selection process via PCC. Specifically, in line 1, we generate each feature vector of the network traffic data. Then using eq. (9), the PCC of each two features will be computed (line 2). In line 3, we check whether the calculated PCCs satisfy the condition. It is noteworthy that instead of finding the minimum feature set for CDM model generation in this article, to reduce the complexity, we only focus on the effectiveness of the feature selection with the PCC approach for the proposed E-FPKD approach. Thus, the following rule is introduced heuristically (line 4): 1) obtain and sort the No. of PCC in $[-1, -0.1] \cup [0.1, 1]$ per feature, 2) retain the features corresponding to the

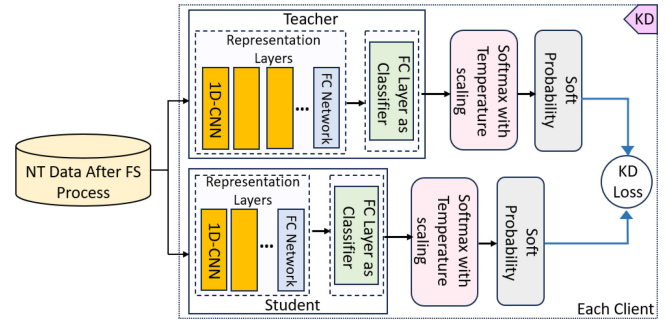


Fig. 5. KD applied to per client in the proposed E-FPKD approach.

top few numbers empirically. Next, the NT data of a client after the feature selection process will be obtained (line 5). Finally, we can get NT data after feature selection in line 6.

Notably, NT data handled by PCC will be used as the input of the proposed E-FPKD approach that will be illustrated next.

B. Availability of Each Client for Joining the FL Process

In realistic, as per [13], it is unrealistic to presume all clients can always offer the services for training. This may affect the willingness of some clients to join in the FL process. Motivated by [13] which takes into account the availability of clients for joining the training process, each client's availability is considered in this work. Concretely, each client can choose whether to participate in the FL process freely. To this end, we assume each client is able to report its availability before joining the FL process to the DSO cloud. The availability per client can be represented as below:

$$\alpha_s = \begin{cases} 1, & \text{if client } s \text{ is available to join,} \\ 0, & \text{otherwise.} \end{cases} \quad (10)$$

C. KD for Each Local Client Construction Toward Cyber-Attack Detection

In this article, since KD in FL is capable of assisting the data heterogeneity issue [17], knowledge distillation (KD) is adopted to build each local client towards cyber-attack detection. Specifically, as mentioned before, identical to [27] which considers using a mentor model and a mentee model to form each client, we consider each client to be constructed with a teacher model (TM) and a student model (SM). Particularly, TM is pre-trained and the knowledge of TM will be distilled to guide SM placed on the same DLES.

In Fig. 5, we demonstrate the KD applied to each client (i.e., DLES s) in the proposed E-FPKD approach, which is a part of Fig. 4. To be specific, each client has two networks: teacher network (TNet) and student network (SNet). TNet is used to generate the teacher model, while SNet is utilized to produce the student model. In addition, the network traffic data after feature selection process will be used as the input of both TNet and SNet. For TNet and SNet, we adopt deep learning to create them. Thus, they both contain the representation layers and classifier.¹ For the sake of simplicity, we use the

¹As mentioned in Section IV-A, the deep learning (DL) model generally consists of the representation layers and the classifier. In this article, we adopt deep learning to build both TNet and SNet. Thus, both TNet and SNet are composed of the representation layers and the classifier.

combination of one-dimensional (1D) convolutional neural network (CNN) and fully connected (FC) network to build the representation layers of TNet and SNet, as shown in Fig. 5. It is noteworthy that the reason for utilizing 1D-CNN is that we represent a network traffic record by a vector. For the classifier, we simply adopt an FC layer. The idea of KD can be the idea of leveraging the TNet's soft probabilities to supervise the SNet [60], where the soft probabilities can be calculated via *Softmax with Temperature scaling* [53] (softmax_T). Accordingly, we consider using softmax_T to convert the output of the classifier into soft probability in this work. Then the soft probability of TNet and SNet will be used to determine the KD loss.

Given the network traffic data at time slot t , x_s^t , the output of TNet per client s can be denoted as $b_s(x_s^t) = [b_s^1(x_s^t), b_s^2(x_s^t), \dots, b_s^k(x_s^t)]$. Here, k represents class k . Then we can calculate the soft probability of TNet, $\varrho_s^k(x_s^t)$, as [53]:

$$\varrho_s^k(x_s^t) = \frac{e^{b_s^k(x_s^t)/\zeta}}{\sum_u e^{b_s^u(x_s^t)/\zeta}}, \quad (11)$$

where $b_s^u(x_s^t)$ denotes u -th elements of vector $b_s(x_s^t)$. Notably, ζ is the temperature and the ζ used in (7) and (11) of this work is the same (identical to [52]). Similarly, the soft probability of SNet, $\widehat{\varrho}_s^k(x_s^t)$, can also be obtained by softmax_T.

In KD, an important point is the KD loss which will be used as a part of the loss function when training the student network of local client. To gain the KD loss (denoted as $\mathcal{L}_{s,SN}^{KD}$), Kullback-Leibler (KL) divergence can be employed. Together with considering the availability of that client (i.e., α_s), according to [52] and eq. (7), it can be calculated as:

$$\mathcal{L}_{s,SN}^{KD} = \alpha_s \frac{1}{|\mathcal{X}_s|} \sum_{t=1}^{|\mathcal{X}_s|} \zeta^2 KL(\varrho_s^k(x_s^t), \widehat{\varrho}_s^k(x_s^t)), \quad (12)$$

where $\mathcal{X}_s$ is the set of total network traffic data of client s ($\mathcal{X}_s = \widehat{\mathcal{X}}_s \cup \widetilde{\mathcal{X}}_s$ as defined in Section IV-B).

In this work, we consider using SNet of each local client to join in the FL process, where the prototypes produced by each student network will be used in the FL process. Thus, in the following, we will illustrate the contents of the prototypes.

D. Prototypes Generation in Each Local Student Network

In this part, we will go through the process regarding the prototypes generation in each local student network, as the prototypes will be utilized in the FL process. The prototypes generation process is shown in Fig. 6. Next, we will illustrate corresponding contents in detail.

As mentioned in Section V-C, *Prototype* is the mean value of the embedding vectors for the instances that belong to class k [18]. Here, an embedding vector is the output of the representation layers for an input instance. Let $\mathbb{N}_s = \{N_s^{(1)}, N_s^{(2)}, \dots, N_s^{(k)}\}$ denote the prototype set generated by the student network of each participating client s (i.e., $\alpha_s = 1$), as shown in Fig. 6 (a part of Fig. 4). Taking the availability of

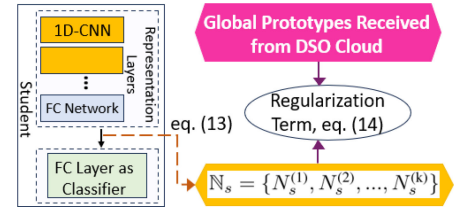


Fig. 6. Prototypes Generation in the local student network.

client s (i.e., $\alpha_s = 1$) into account, as per [18], the prototype $N_s^{(k)}$ of a participating client s can be represented by,

$$N_s^{(k)} = \alpha_s \frac{1}{|\mathcal{D}_{s,k}|} \sum_{(x_s^t, y_s^t) \in \mathcal{D}_{s,k}} \Omega_s(\theta_s; x_s^t), \quad (13)$$

where $\mathcal{D}_{s,k}$ is composed of the training network traffic data and labels that belong to the k^{th} category, which is the subset of the local network traffic dataset ($\mathcal{D}_s = \{(x_s^1, y_s^1), (x_s^2, y_s^2), \dots, (x_s^t, y_s^t)\}, x_s^t \in \mathcal{X}_s, y_s^t \in \mathcal{Y}_s$). $\Omega_s(\theta_s; x_s^t)$ is the embedding function for the representation layer with the parameter θ_s . After obtaining the prototypes in each local client, those prototypes will be transmitted to the DSO cloud for aggregation to produce the global prototypes. The aggregation process will be explained in Section VI-F.

During student network training, according to [32], in order to achieve a better representation net, the regularization term for the student network, $\mathcal{L}_{s,SN}^R$, can be considered and this regularization term will be added to the loss function of the local student network. Specifically, the regularization term can be obtained through punishing the distance between two types of prototypes (i.e., local prototype and global prototype) [32]. Particularly, it is noteworthy that for each local SNet, we use the prototypes generated in the current round and the global prototypes obtained in the previous round to calculate the regularization term which can be given as below [18]:

$$\mathcal{L}_{s,SN}^R = \sum_k distance(N_s^{(k)}, \overline{N}^{(k)}), \quad (14)$$

where $\overline{N}^{(k)}$ is the global prototype for class k which will be given in Section VI-F. For the distance metric, $distance(\cdot, \cdot)$, it can be gained by using *L1*, *L2* distance and the forth [32].

E. Local Objective of Each Client

Naturally, many loss functions have been designed by researchers to raise classification accuracy [61]. Loss function optimization is usually expressed as a minimization problem [62]. Motivated by these, to maximize the overall detection correctness of all prosumers (ODC) (i.e., (5)), minimizing the loss function for each local SNet is considered. Before defining the local loss function of this work, it is valuable to mention that multiple types of cyberattacks may exist in the real world. Thus, without loss of generality, we consider both binary and multi-class classification in this work. Particularly, we only consider two classes: malicious (i.e., anomaly) class and benign (i.e., normal) class for binary classification, while we consider a normal class and various

cyber-attacks classes for multi-class classification. In the following, we will illustrate the considered local loss function in detail.

Concretely, the local loss function of student network per client s (denoted as $\mathcal{L}_{s,SN}$) is defined as the linear combination of the loss for the supervised learning of SNet (denoted as $\mathcal{L}_{s,SN}^{SL}$) and the regularization term. Regarding $\mathcal{L}_{s,SN}^{SL}$, we define it as the linear combination of the binary cross entropy (BCE) [63] loss and KD loss for the binary classification, and the linear combination of the cross entropy (CE) loss [64] and KD loss for the multi-class classification.² For the readability of this paper, we use $\mathcal{L}_{s,SN}^{\text{entropy}}$ to represent BCE loss and CE loss. Namely, $\mathcal{L}_{s,SN}^{\text{entropy}}$ is BCE loss for binary classification, and is CE loss for multi-class classification. As a result, we give the loss function of the student network of client s as below:

$$\begin{aligned}\mathcal{L}_{s,SN} &= \mathcal{L}_{s,SN}^{SL} + \gamma \mathcal{L}_{s,SN}^R \\ &= \psi \mathcal{L}_{s,SN}^{\text{entropy}} + (1 - \psi) \mathcal{L}_{s,SN}^{KD} + \gamma \mathcal{L}_{s,SN}^R, \quad (15)\end{aligned}$$

where ψ and γ are hyperparameters. Consequently, the local objective of each local student network can be given as below:

$$\min_{\theta_s, \vartheta_s, N_s^{(k)}} \mathcal{L}_{s,SN}, \quad (16)$$

where ϑ_s denotes the parameter of the classifier. As for the teacher network, to pre-train it, we simply use BCE loss for binary classification, and CE loss for multi-class classification.

F. Availability-Based Aggregation Process (Global Side) for Cyber-Attacks Detection

To handle non-IID data issue, in addition to KD, we also employ the prototype aggregation technique [18] at each round to assist non-IID data processing. In order to facilitate future applications, we also take into account aggregating the local models received from the SNet of each participating client for global model generation. The details will be explained next.

Prototype aggregation at each round. After DSO cloud receives the prototypes generated by the student network of each participating client at each round, the prototype aggregation process will be occurred. Considering the availability of each client α_s , according to [18] and eq. (8), the global prototype (denoted as $\bar{N}^{(k)}$) for class k can be obtained by,

$$\bar{N}^{(k)} = \frac{1}{|\mathcal{M}_k|} \sum_{s \in \mathcal{M}_k} \frac{\alpha_s |D_{s,k}|}{\sum_{s \in \mathcal{M}_k} |D_{s,k}|} N_s^{(k)}. \quad (17)$$

As denoted before, $\mathcal{M}_k$ is a set of the participating clients that own class k . $|D_{s,k}|$ is the size of $D_{s,k}$.

Model aggregation at the final round for future application. Through the training process of the E-FPKD approach, although the heterogeneous data issue can be handled, there is another problem that needs to be solved if the aggregation is only based on the prototypes. The problem

is that with prototype aggregation, each client will have its own personal model. Thus, if a new client joins the process, even though the global prototypes can be downloaded from the DSO cloud, it requires the newly joined client to do the fine-tuning process by adjusting its local prototypes to match the downloaded global prototypes as much as possible. To tackle this problem, to be simplified, firstly, we assume each participating client agrees with using the same structure. Secondly, we consider using the prototype aggregation for training in all rounds, and then, different from [18], we introduce model aggregation in the final round to generate the global model. Then we consider using this aggregated global model for application (i.e., cyber-attack detection). In such a way, since the obtained global model can be directly utilized by each client, if there is a newly joined client, the fine-tuning process can be eliminated for that client. Let ω_s indicate the model of the student network of client s that consists of the representation layer parameter θ_s and the classifier parameter ϑ_s . Namely, $\omega_s = (\theta_s, \vartheta_s)$. Then considering the availability per client simultaneously, as per [50], the global model ω in the final round (denoted as Q) can be calculated as,

$$\omega = \sum_{s \in \mathcal{S}} \alpha_s \frac{|\mathcal{D}_s|}{\hat{H}} \omega_s, \quad (18)$$

where $\hat{H} = \sum_{s \in \mathcal{S}} \alpha_s |\mathcal{D}_s|$ is the total size of dataset owned by the participating clients. To obtain ω_s , as per [48], one step of the gradient descent can be performed on the current model by using the local data. Concretely, ω_s can be calculated by $\omega_s \leftarrow \omega_s - \hat{\eta} \nabla \mathcal{L}_{s,SN}$ [48]. $\hat{\eta}$ denotes the learning rate [48].

Global objective. For global objective, identical to FedProto [18] which considered minimizing the summation of loss towards the local learning tasks of all clients, we also adopt the same manner. Taking the per client's availability α_s into consideration, according to [18], the global objective of this work can be defined as,

$$\begin{aligned}& \argmin_{\{\bar{N}^{(k)}\}_{k=1}^K} \sum_{s=1}^{|S|} \frac{\alpha_s |\mathcal{D}_s|}{U} \mathcal{L}_{s,SN}^{SL} \\ & + \gamma \cdot \sum_{k=1}^K \sum_{s=1}^{|S|} \frac{\alpha_s |\mathcal{D}_{s,k}|}{U_k} \text{distance}(\bar{N}^{(k)}, N_s^{(k)}). \quad (19)\end{aligned}$$

Here, K is the total number of classes. Since one prototype corresponds to one category, K can also be recognized as the total number of global prototypes. U is leveraged to represent the overall number of all participating clients' instances, where $U = \sum_k^K U_k$ and $U_k = \sum_{s \in \mathcal{M}_k} |D_{s,k}|$ denotes the number of instances belonging to class k over all participating clients. As we define the supervised learning loss ($\mathcal{L}_{s,SN}^{SL}$) as $\mathcal{L}_{s,SN}^{SL} = \psi \mathcal{L}_{s,SN}^{\text{entropy}} + (1 - \psi) \mathcal{L}_{s,SN}^{KD}$, we can rewrite (19) as below:

$$\begin{aligned}& \argmin_{\{\bar{N}^{(k)}\}_{k=1}^K} \sum_{s=1}^{|S|} \frac{\alpha_s |\mathcal{D}_s|}{U} [\psi \mathcal{L}_{s,SN}^{\text{entropy}} + (1 - \psi) \mathcal{L}_{s,SN}^{KD}] \\ & + \gamma \cdot \sum_{k=1}^K \sum_{s=1}^{|S|} \frac{\alpha_s |\mathcal{D}_{s,k}|}{U_k} \text{distance}(\bar{N}^{(k)}, N_s^{(k)}). \quad (20)\end{aligned}$$

²Binary cross entropy is the loss function used in binary classification [63], where the label is defined as 0 and 1. Cross entropy loss is one of the most widely utilized loss function in the process of deep neural network model training, especially for multi-class classification problems [65].

G. Rule-Based Method for Cyber-Attacks Intervention

The global model generated by the model aggregation at the final round will be utilized for cyber-attack detection in the future application stage. In this stage, after detecting the cyber attacks, a cyber-attacks intervention process on each dedicated local edge server (DLES) will be performed, aiming to protect the considered prosumer-based EVCSs from being attacked by those detected cyber attacks.

To do this, *rule-based method* [36] is introduced. Concretely, to express the rule, a basic method called “if...then” [36] is adopted, which means if “condition” is true, then “conclusion” will be performed. Inspired by [14] which considers triggering a preventive event in the edge service when DDoS attack is identified, we consider the following rules:

- If the detected result belongs to the malicious category, then the preventive event [14] will be triggered at each DLES automatically. For simplicity, we consider blocking the malicious NT directly as the preventive event.
- If the detected result is normal, then we do not kick-start any preventive event at DLES.

H. Algorithm for the Proposed E-FPKD Approach Towards Cyber-Attacks Prevention

Algorithm 2 shows global model generation process in E-FPKD approach, while Algorithm 3 shows the proposed E-FPKD for cyber-attack prevention in future application stage.

In Algorithm 2, in line 1, we use Algorithm 1 to execute feature selection process. In line 2, we pre-train the TM of each client. Lines 3-17 show the process in the DSO cloud. Concretely, in lines 3-5, we initialize the global model, the student network model per client and the global prototype set for all classes. Next, for each round (line 6), we check the availability of each client and get the participating clients in line 7. Then for each participating client s (line 8), the global prototype set will be submitted to it (line 9). In line 10, if it is not the final round (the final round is denoted as Q), only the prototypes of SNet of the participating client s will be received by DSO cloud. Otherwise, both the prototypes and SNet model of client s will be received. Those prototypes and the local student network model are generated in lines 18-33. After receiving the prototypes from all participating clients, the global prototype will be updated with eq. (17) as shown in line 12. Finally, in lines 13-15, we check whether the current round is the ultimate round, if it is (line 13), we will aggregate the student network models received from all participating clients to generate the global model (line 14) which will be used to detect cyber-attacks.

In Algorithm 3, we list the process of applying the obtained global model for cyber-attack prevention in the future application stage. In line 1, we do feature selection for new NT data via Algorithm 1. Then for each joined client (line 2), cyber-attack prevention will be performed in lines 3-5. Concretely, we load the global model obtained by Algorithm 2 to each joined client in line 2. In line 3, the detection results will be obtained. Based on the detection results, the rule-based method will be performed to block the malicious NT data in line 4.

Algorithm 2 Global Model Generation in E-FPKD for Cyber-Attack Detection

Input: Each client's dataset: $\mathcal{D}_s$, client availability: α_s , $s \in S$, training round: Q

Output: Global model ω

```

1: Feature selection via Algorithm 1
2: Pre-train TM per client
Process in the DSO cloud:
3: Initialize global model  $\omega$ 
4: Initialize each client's student network model:  $\omega_s \leftarrow \omega$ 
5: Initialize the set of global prototype  $\{\bar{N}^{(k)}\}$  for all classes
6: for each round  $q = 1, 2, \dots, Q$  do
7:   Get participating client by checking  $\alpha_s$ 
8:   for each participating client  $s$  in parallel do
9:     Submit the global prototype set  $\{\bar{N}^{(k)}\}$  to the participating client  $s$ 
10:     $\{N_s^{(k)}\} \leftarrow \text{ClientUpdate}(s, \{\bar{N}^{(k)}\}, q)$ , if  $q \neq Q$ ,
        $\omega_s, \{N_s^{(k)}\} \leftarrow \text{ClientUpdate}(s, \{\bar{N}^{(k)}\}, q)$ , if  $q = Q$ 
11:   end for
12:   Update global prototype via eq. (17):  $\{\bar{N}^{(k)}\}$ 
13:   if  $q == Q$  then
14:     Generate global model  $\omega$  by aggregating each local student network model  $\omega_s$ 
15:   end if
16: end for
17: return the obtained global model  $\omega$ 

```

ClientUpdate($s, \{\bar{N}^{(k)}\}, q$):

```

18: for each epoch do
19:   for batch  $(x_s, y_s) \in \mathcal{D}_s$  do
20:     Get embedding vectors from representation layer
21:     Compute KD loss:  $\mathcal{L}_{s,SN}^{KD}$ 
22:     Calculate BCE/CE loss:  $\mathcal{L}_{s,SN}^{\text{entropy}}$ 
23:     Calculate regularization term:  $\mathcal{L}_{s,SN}^R$ 
24:     Calculate local loss:  $\mathcal{L}_{s,SN}$ 
25:      $\omega_s \leftarrow \omega_s - \hat{\eta} \nabla \mathcal{L}_{s,SN}$ 
26:   end for
27: end for
28: Get the prototype set  $\{N_s^{(k)}\}$  via eq. (13)
29: if  $q \neq Q$  then
30:   return  $\{N_s^{(k)}\}$ 
31: else
32:   return  $\omega_s, \{N_s^{(k)}\}$ 
33: end if

```

Remark 1: Trade-off between Security and Computational Efficiency.

The proposed E-FPKD approach is designed based on FL. It is possible to support EVCSs with large-scale prosumers. When the No. of prosumers increases, we merely need to raise the No. of the clients in the proposed method. Like FL, accessing the local data owned by each client is not required by the server. Thus, compared with centralized manner, the data privacy issue caused by centrally gathering data can be protected to a certain extent. Hence, for an environment

Algorithm 3 E-FPKD for Cyber-Attack Prevention (Future Application Stage)

-
- 1: Feature selection via **Algorithm 1**
 - 2: **for** each joined client **do**
 - 3: Load the global model ω obtained by **Algorithm 2**
 - 4: Get detection results by executing the global model
 - 5: Block the malicious NT data via the rule-based method
 - 6: **end for**
-

with high data privacy requirements, the proposed method can be preferred over centralized methods. As for the independent method (training model individually), the average performance across all the clients may be degraded, compared with the proposed method. Because of the existence of the data non-IID issue, some clients may not have abundant data. This can affect the overall performance, because insufficient data can hinder the performance as per [66]. Thus, there is a trade-off between the proposed method and the independent methods. Regarding the proposed method, the additional computational overhead may happen at each client during teacher model (TM) pre-training phase. However, this only occurs in the beginning. Because we do not need to train TM every round, thereby, the computation overhead of the proposed method can be still smaller than the one with training TM every round.

In addition, we introduce prototype aggregation (PA) in each round, while only in the final round, both PA and model aggregation (MA) are used. Owing to this manner, the risk of inferring original input data may be reduced, compared with using MA in each round. Because the input data is able to be inferred through the given model [67]. As for each prototype, it is an average value of the embedding vectors and a prototype corresponds to a class, which can increase the difficulty of reasoning the data structure (e.g., data vector length). Accordingly, since we only adopt MA in the final round, we can infer that the data leakage risk of the proposed method can be smaller than using MA per round. Moreover, since KD and prototype aggregation are beneficial for non-IID data processing, the proposed method is advantageous in processing heterogeneous data. As a result, considering the security issue (data leakage) and the potential of handling non-IID data, it is reasonable to employ the proposed method for cyber-attack prevention.

Time Complexity (TC). The proposed E-FPKD approach is an FL-based approach composed of a server (i.e., DSO cloud) and multiple clients, where each client consists of TNet and SNet. In particular, TNet is pre-trained and SNet is used for training. We adopt the combination of CNN and fully connected (FC) network to form each TNet and SNet. For TNet, same as [68], we calculate its time complexity by summing up the time complexity of CNN and FC networks. Specifically, the TC of a convolutional layer of TNet can be given as: $O(G^{ne} G^{nf} G^{ks} (G^{ts} - G^{ks} + 1))$ [68], where G^{ne} , G^{nf} , G^{ks} and G^{ts} denote the number of elements of the input vector, the number of CNN filters, kernel size and the number of time steps (time slots in this work) [68]. As shown in [69], the overall time complexity of all the convolutional layers is given

by summarizing the TC of each convolutional layer. Thus, the time complexity of all the convolutional layers for each TNet is $O(\sum_{e_t} G^{ne} G^{nf} G^{ks} (G^{ts} - G^{ks} + 1))$. Here, e_t denotes $(e_t)^{th}$ convolutional layer of TNet. As for FC network, the time complexity can be given as $O(\sum_{\hat{e}_t} G_{\hat{e}_t-1}^{nu} G_{\hat{e}_t}^{nu})$ [50], where $G_{\hat{e}_t-1}^{nu}$ is the number of the neural units (NUs) in the FC layer $\hat{e}_t - 1$ and $G_{\hat{e}_t}^{nu}$ indicates the number of the NUs in the FC layer $\hat{e}_t$. Accordingly, the time complexity of each client's TNet is $O(\sum_{e_t} G^{ne} G^{nf} G^{ks} (G^{ts} - G^{ks} + 1) + \sum_{\hat{e}_t} G_{\hat{e}_t-1}^{nu} G_{\hat{e}_t}^{nu})$. In contrast with TNet, during training, SNet per client s needs to compute the local prototypes. Thus, for SNet, the time complexity contains the time complexity of both the network and local prototypes generation. Considering each SNet generates K_s prototypes, the time complexity of each SNet at most is $O(\hat{E}(\hat{Z} + K_s |\mathcal{D}_s|))$ [70]. Here, $\hat{E}$ denotes the local iteration, and $\hat{Z}$ is the time complexity of the backbone network (i.e., CNN + FC network). Similar as TNet, $\hat{Z}$ can be given as $O(\sum_{e_s} G^{ne} G^{mf} G^{ks} (G^{ts} - G^{ks} + 1) + \sum_{\hat{e}_s} G_{\hat{e}_s-1}^{nu} G_{\hat{e}_s}^{nu})$, where e_s and $\hat{e}_s$ represent the $(e_s)^{th}$ convolutional layer and $(\hat{e}_s)^{th}$ FC layer of SNet, respectively. $G_{\hat{e}_s-1}^{nu}$ and $G_{\hat{e}_s}^{nu}$ separately denote the NUs in the FC layer $\hat{e}_s - 1$ and in the layer $\hat{e}_s$ of FC network. In regard to the server, we perform prototype aggregation in each round and both prototype and model aggregation in the final round. For model aggregation, FedAvg [48] is adopted in this work. Let M denote the number of participating clients in each round, the time complexity for server to perform FedAvg is bounded as $O(M)$ [71]. As for prototype aggregation, in each round, the time complexity of updating the global prototype through prototype aggregation is no more than $O(KM)$ [70]. Accordingly, the TC of server is $O(M + KM)$ in the final round, while TC of server is $O(KM)$ in other round. Thus, the overall TC of the proposed method for SNet per client is $O(\hat{E}(\hat{Z} + K_s |\mathcal{D}_s|) + (KM + M))$ [70] in the final round. As for other round, the overall time complexity for SNet of each client is $O(\hat{E}(\hat{Z} + K_s |\mathcal{D}_s|) + KM)$.

VII. EXPERIMENTAL RESULTS AND ANALYSIS

In this section, we conduct experiments for the proposed method, where both quantitative and qualitative analyses are considered. The detailed contents will be shown from Sections VII-A–VII-C. Notably, for cyber attack intervention, we simply consider directly blocking the malicious NT by a basic rule-based method when an attack is detected. Accordingly, for simplicity, the evaluation for this process is ignored.

A. Experimental Setup

1) *Model Setup:* In this article, we consider the teacher network consists of three 1D-convolutional layers with two fully connected (FC) layers. Particularly, the number of channels for the input of the convolutional layers is separately 1, 512 and 1024, while the number of channels generated by the convolutional layers are 512, 1024, and 2048, respectively. Each convolutional layer is considered to connect with a Batch Normalization and a ReLU activation function. As for the first FC layer, the input unit is the features' number for a record $\times$

TABLE IV
PARAMETER SETTINGS

Parameter Description	Value
No. of clients	10 [74], 20, 50
Dirichlet parameter	0.5, 0.9 [75], 5
Training Round	
NSL-KDD dataset (N)	{10, 20, 50, 100}
UNSW-NB15 dataset (U)	{5, 10, 20, 50}
IoTID20 dataset (I)	{5, 10, 15, 20}
Teacher Network	
Learning rate	0.001
Batch size	32 (N) / 512 (U) / 128 (I)
Local epoch	5
No. of channels for the input of each convolutional layer	{1, 512, 1024}
No. of channels generated by each convolutional layer	{512, 1024, 2048}
Input unit for the 1 st FC layer	No. of features for a record $\times$ 2048
Output unit for the 1 st FC layer	512
Input unit for the 2 nd FC layer	512
Output unit for the 2 nd FC layer	No. of classes
Optimizer	Adam
Student Network	
Initial learning rate	0.0001
Learning rate decay	0.97 [76]
Batch size ($\{N, U, I\}$ dataset)	32 (N) / 512 (U) / 128 (I)
Local epoch	5
Temperature ($\{N, U, I\}$ dataset)	0.5 (I) [77] / 0.1 (U, I) [78]
Hyperparameters ψ and γ	0.1 (ψ), 1 (γ)
No. of input channels for each 1D-convolutional layer	{1, 64}
No. of output channels for each 1D-convolutional layer	{64, 128}
Input unit for the 1 st FC layer	No. of features for a record $\times$ 128
Output unit for the 1 st FC layer	64
Input unit for the 2 nd FC layer	64
Output unit for the 2 nd FC layer	No. of classes
Optimizer	SGD

2048, and the output unit is 512. For the second FC layer, the input unit is 512, while the output unit is the number of classes. In the case of the student model, we consider it owns two 1D-convolutional layers and two FC layers. Specifically, regarding the convolutional layers, the number of input channels for each layer are separately 1 and 64, while the output units are 64 and 128, respectively. Besides, the input for the first FC layer is *the number of features for a record $\times$ 128*, and the corresponding output unit is 64. The second FC layer's input unit is 64 and the output unit is the number of classes. In addition, in regard to the optimizers, we adopt Adam for the teacher network and SGD for the student network. Additionally, identical to [72] which considers randomly initializing the weight of the deep neural network (DNN)-based network at the first iteration, we consider initializing the weights of TNet and SNet randomly in the beginning round. Moreover, regarding the availability of the clients, inspired by [73] which considers randomly selecting EV charging stations to join the FL process in each round, we also adopt the stochastic selection for determining the availability of the clients (i.e., DLEs of prosumers) in each round. The settings of the parameters are summarized in Table IV. Notably, for datasets, we adopt NSL-KDD [37], UNSW-NB15 [38], [39], [40], [41], [42], and IoTID20 dataset [43] (describe in Section VII-A3). *The features' number for a record* depends on the dataset, which will be illustrated in Section VII-A4.

2) *Baselines*: To evaluate the effectiveness of the proposed method, we compare it with the following benchmarks: 1) *the E-FPKD approach without feature selection (FS)*, where we remove FS but keep other parts same as the proposed E-FPKD method. 2) *The E-FPKD approach without learning rate (LR) decay*, where we discard the LR decay (0.97 [76]) per round but embrace other parts of the proposed method. Particularly, we adopt $LR = LR \times 0.97^{q-1}$ to calculate LR in q^{th} round. 3) *The E-FPKD approach with DNN-based representation layers*, where DNN is used to create the representation layers. In particular, for handling the NSL-KDD dataset, the network per client contains two FC layers as representation layers, and one FC layer as classifier. The input units for each FC layer are separately *the number of features for a record*, 128 and 64, while the output units for each FC layer are 128, 64 and *No. of classes*, respectively. To address the UNSW-NB15 and IoTID20 datasets, we consider using two FC layers, where one layer is regarded as representation layer, and the other is treated as classifier. The input units for each layer are *the number of features for a record* and 32, respectively. The output units per layer are separately 32 and *No. of classes*. 4) *Independent CNN*, where we apply CNN for each client independently (the CNN is the same as the SNet adopted by the proposed method); 5) *Independent KD*, where KD technique is leveraged for each client independently (the KD is identical to the proposed method); 6) *FedKD*: where we retain the KD part of the proposed method and use model aggregation in each round. 7) some other FL-based framework: *FedAvg* [48], *FedProx* [79], *FedProto* [18], *pFedSD* [25], *FedAU* [44], *FedExp* [45], *FedGKD* [80]. For these methods, we embrace the student network of the proposed method to form each client. Next, we will discuss the **limitations of baseline methods and how to deal with them with the proposed E-FPKD approach**.

In regard to E-FPKD without feature selection (FS) method, lack of FS may result in irrelevant features in the input data. Large irrelevant features will cause serious issues for the existing machine-learning approaches [81]. Thus, since FS is capable of excluding irrelevant features [15], we adopt FS in this work. With respect to the method of E-FPKD without learning rate (LR) decay, the LR of the network will continue to be fixed. According to [82], if LR is too small, the model training process will converge slowly; if LR is too large, the model performance will degrade. Hence, to solve this issue, same as [82], we adopt LR decay in this work, which will make the LR vary each round. When it comes to the method of E-FPKD with DNN-based representation layers (RLs), DNN is used as the RLs. For this manner, huge amount of data may be required for training. This is because to achieve the target accuracy, DNN needs to use large input data to train the model [83]. However, some prosumers may lack data, thus, in such a situation, it will become inappropriate to use DNN. As per [84], adding the convolutional layer can increase the network nonlinearity, which can help increase the accuracy. Therefore, instead of DNN, we employ the convolutional layers in this work. In the case of FedKD, FedAvg [48], FedProx [79], pFedSD [25], FedAU [44], FedExp [45] and

FedGKD [80], all of these methods utilize model aggregation method. With model aggregation, as the input data is capable of being inferred through a given model [67], the input data inference problem needs to be focused. For FedProto [18], despite FedProto leverages prototype aggregation rather than model aggregation, when a new client joins, fine-tuning process is required [18]. To balance the model inference and fine-tuning issues, in the proposed method, we conduct prototype aggregation in each round, and model aggregation in the final round to generate the global model for future use.

3) *Datasets Description*: To verify the goodness of the proposed approach, we use three datasets including **NSL-KDD dataset** [37] (i.e., KDDTrain+.csv and KDDTest+.csv), **UNSW-NB15 dataset** [38], [39], [40], [41], [42] (UNSW_NB15_training-set.csv and UNSW_NB15_testing-set.csv are adopted), and **IoTID20 dataset** [43].

NSL-KDD dataset [37] KDDTrain+.csv is employed for global model generation and KDDTest+.csv is used in application stage. This dataset is a new version of the KDD dataset that is composed of the chosen records from the full KDD dataset [85]. As described in [86], for the NSL-KDD dataset, it contains 43 features, where 41 features are the input traffic features and the other two features are the label of the category (i.e., normal or attack) and the score used to indicate the severity of the attack. For simplicity, the score feature is abandoned in this article. In addition, this dataset is labeled under normal and four categories, viz., DoS, probe, U2R and R2L. It is worth noting that the NSL-KDD dataset has several advantages. As per [85], those advantages are given below: 1) in the training dataset, there are no redundant records, 2) for the test dataset, it does not contain duplicate records, 3) in both the training and test datasets, the number of records is reasonable (i.e., KDDTest+.csv has 22544 records, while KDDTrain+.csv has 125973 records). As aforementioned, we consider both binary and multi-class classification. Thus, in *binary classification*, we regard DoS, probe, U2R, and R2L as anomaly class, and normal as normal class. In *multi-class classification*, we consider five classes, i.e., DoS, probe, U2R, R2L and normal.

UNSW-NB15 dataset [38], [39], [40], [41], [42] UNSW-NB15 data is generated by [38] from the real-world network flows [87]. As highlighted before, UNSW_NB15_training-set.csv and UNSW_NB15_testing-set.csv are adopted for evaluation process. In particular, UNSW_NB15_training-set.csv is utilized for producing the global model, while UNSW_NB15_testing-set.csv is adopted for application stage. For these two files, there are 175,341 and 82,332 records for training part and testing part, respectively. In the case of the labels for two files, 10 labels are included, namely, Analysis, Backdoor, DoS, Exploits, Fuzzers, Generic, Reconnaissance, Shellcode, Worms, and Normal. Thus, in *binary classification*, the record with the labels of Analysis, Backdoor, DoS, Exploits, Fuzzers, Generic, Reconnaissance, Shellcode, and Worms are treated as anomaly category, while the records with Normal label are regarded as normal category. In *multi-class classification*, we consider these 10 labels as 10 categories for detection.

IoTID20 dataset [43] IoTID20 dataset was proposed by [43], which contains a huge number of the general network features as well as flow-based features [88]. For this dataset, it is produced from a real-time scenario by IoT device [89] and can be used for detecting the anomalous activity in the IoT networks [88]. Besides, it contains 86 columns and 625,783 records (i.e., rows) [89]. Considering this dataset contains duplicate records, same as [89], we consider removing those duplicate records. Consequently, only 461,696 [89] records are left. In addition, for IoTID20, there exists several *infinite values* [90]. It can be known from [90] that we can remove those infinite values. Hence, for simplicity, the records with infinite value are discarded in this paper. Additionally, some features in IoTID20 dataset belong to nominal type, which are highly regarding the testbed's network configuration [91]. Those features include $\{Flow_ID, Src_IP, Dst_IP, Timestamp\}$ [91]. As mentioned in [91], those features can be discarded. Thus, we get rid of those features for convenience. For global model generation and testing in the application phase, we consider using around 70% and 30% data, respectively. In IoTID20 dataset, 3 features that are related to label are provided, where those features are named *Label*, *Cat*, and *Sub_Cat*, respectively. To be specific, the *Label feature* is composed of $\{Anomaly, Normal\}$ [90] these two classes, while the *Cat feature* consists of $\{DoS, Mirai, MITM\ ARP\ Spoofing\ (MAS), Scan, Normal\}$ [90] these five classes. As for the *Sub_Cat feature*, it consists of more classes. Thus, to be convenient, the *Label feature* is used for binary classification in this work, while the *Cat feature* is leveraged for multi-class classification.

4) *Data Preprocessing via Feature Selection (FS)*: The original NSL-KDD and UNSW-NB15 datasets have training and test parts. For simplicity, we perform FS process with Pearson Correlation Coefficient (PCC) for the training part of NSL-KDD and UNSW-NB15. For IoTID20, FS process is executed for the whole dataset. As mentioned in Section VI-A, we consider PCC in the range of $[-1, -0.1] \cup [0.1, 1]$ for feature selection. From Fig. 7 to Fig. 9, we list the number of PCC values for each feature versus other features with $0.1 \leq PCC \leq 1$ or $-1 \leq PCC \leq -0.1$. Notably, the *Label feature* is excluded. Since we only focus on the effectiveness of the feature selection with the PCC approach for the proposed E-FPKD method rather than finding the minimum feature set, for simplicity, we retain the features with the larger number of PCC values in the range of $[-1, -0.1] \cup [0.1, 1]$. Empirically, we select the top 22 features with the largest number of PCC values that meet the conditions for NSL-KDD dataset. Similarly, the top 27 features of UNSW-NB15 dataset are selected, while the top 40 features of IoTID20 dataset are chosen. The name of those selected features can be found in Fig. 7, Fig. 8 and Fig. 9, respectively.

5) *Scenario and Condition*: As the NT data per prosumer may exist non-IID phenomenon, and various levels of data heterogeneity may pose challenges for constructing the cyber attack prevention mechanism in real-life deployments, it is imperative to investigate a mechanism that can shed light on the effectiveness across different heterogeneity degrees towards real-world scenarios. To this end, for the sake of

F1 score is used to represent the harmonic average value of the precision and the recall [98], which can be given as:

$$\Upsilon_{F1_score} = \frac{2\Upsilon_{Prec}\Upsilon_{RCL}}{\Upsilon_{Prec} + \Upsilon_{RCL}} = \frac{2TP}{2TP + FP + FN}. \quad (27)$$

In terms of the **average F1 score** $\bar{\Upsilon}_{F1_score}$, it is defined as the mean F1 score of all participating clients. Hence, we can give the following calculation manner:

$$\bar{\Upsilon}_{F1_score} = \frac{\sum_{s \in \mathcal{S}} \alpha_s \cdot \Upsilon_{F1_score}^s}{\sum_{s \in \mathcal{S}} \alpha_s}, \quad (28)$$

where the $\Upsilon_{F1_score}^s$ denotes the F1 score of client s that is capable of being obtained through eq. (27).

False alarm rate (FAR) is used to measure the proportion of normal instances that are detected as attacks. This can be known through [97]. The FAR is computed by [97],

$$\Upsilon_{FAR} = \frac{FP}{TN + FP}. \quad (29)$$

② **Multi-class Classification.** For simplicity, we only use accuracy to evaluate the proposed solution in multi-class classification. The accuracy (Υ_{Acc}) is defined as the quotient between the number of the correct detected results (denoted as Ξ_{cdr}) and the overall number of network traffic data in a dataset (represented as $\Xi_{overall}$).

$$\Upsilon_{Acc} = \frac{\Xi_{cdr}}{\Xi_{overall}}. \quad (30)$$

2) *Efficiency Comparisons via QPMs:* According to [92], [93], [94], [95], we summarize the adopted QPMs in Section VII-B1 for evaluating the efficiency of the proposed method. Thus, in this part, we will conduct efficiency comparisons between the proposed method and baselines via considered QPMs.

Various Scenarios. In Table VI, we perform the proposed E-FPKD approach on three datasets for binary and multi-class classification, where we choose different parameter for *Dirichlet Distribution* (i.e., $\delta = \{0.5, 0.9, 5\}$). It is noted that the smaller δ means higher data heterogeneity [75]. Through Table VI, it can be observed that for binary classification over the considered three datasets, the higher the data heterogeneity (i.e., smaller δ), the higher the accuracy achieved by the proposed method. However, when it comes to multi-class classification, it can be seen that for both UNSW-NB15 [38], [39], [40], [41], [42] and IoTID20 [43] datasets, the accuracy increases as δ decreases. Whereas, concerning the NSL-KDD dataset [37], when δ changes from 5 to 0.9, the accuracy rises, but when it drifts from 0.9 to 0.5, the accuracy declines. Thus, we can conjecture that the proposed method can be adapted to Non-IID data environment within a certain range. For the convenience of evaluating the proposed method, we select $\delta = 0.9$ in the following.

Scalability. Similar to [100] which evaluates the scalability of their proposed solution by considering *various number of clients on two datasets*, in this work, we assess the scalability of the proposed approach by considering the *various scale of clients on three datasets*. Particularly, we consider both binary and multi-class classification scenarios.

TABLE VI
ACCURACY ACHIEVED BY THE PROPOSED E-FPKD WHEN CONSIDERING DIFFERENT δ FOR DIRICHLET DISTRIBUTION

Dataset	δ	Accuracy	
		Binary Classification	Multi-class Classification
NSL-KDD	0.5	0.7972	0.6877
	0.9	0.7629	0.7170
	5	0.7295	0.6802
UNSW-NB15	0.5	0.7131	0.6286
	0.9	0.6834	0.6119
	5	0.5832	0.5459
IoTID20	0.5	0.9688	0.7725
	0.9	0.9581	0.7710
	5	0.9543	0.7648

TABLE VII
ACCURACY ACHIEVED BY THE PROPOSED E-FPKD APPROACH WHEN CONSIDERING VARIOUS NUMBERS OF CLIENTS

Classification	# Clients	Accuracy		
		NSL-KDD	UNSW-NB15	IoTID20
Binary	10	0.7629	0.6834	0.9581
	20	0.7372	0.6243	0.9228
	50	0.7299	0.5843	0.8849
Multi-class	10	0.7170	0.6119	0.7710
	20	0.6627	0.5470	0.7464
	50	0.6219	0.5371	0.7107

For each scenario, we consider three cases which consists of 10, 20 and 50 clients, respectively, where $\delta = 0.9$ is adopted for Dirichlet Distribution. In Table VII, we exhibit the accuracy realized by the proposed method. For instance, for multi-class classification, considering 10 clients, the accuracy for NSL-KDD [37], UNSW-NB15 [38], [39], [40], [41], [42] and IoTID20 [43] datasets (test part) are 0.7170, 0.6119 and 0.7710, respectively. Through Table VII, it can be known that when the number of clients is on an upward trend, the accuracy is on a downward trend. The reasons for this phenomenon are speculated as follows: 1) the total data quantity of those three datasets is fixed regardless of the number of clients, 2) additionally, for each round, it may be not all clients are willing to participate in model generation process. As observed in Table VII, the proposed approach can support varying scale of clients in both binary and multi-class classification. Thus, we can infer that the proposed method has the characteristic of scalability. For simplicity, next we will consider 10 clients for further evaluation.

In Table VIII, we compare the performance of the proposed E-FPKD approach with other three methods, i.e., 1) the E-FPKD without feature selection (FS) method, 2) the E-FPKD without learning rate (LR) decay method, 3) the E-FPKD framework but adopting DNN as the representation layers (RL), in terms of binary and multi-class classification. It is noteworthy that we comprehensively employ the FS, LR decay, and CNN-based representation layers in our proposed method. Through Table VIII, it can be observed that the proposed method can achieve the highest accuracy among the

TABLE VIII
ACCURACY FOR THREE DATASETS (TEST PART) ACHIEVED BY VARIOUS METHOD. BOLD NUMBERS INDICATE THE BEST RESULTS

Classification	Dataset	Accuracy			
		Without FS	Without LR Decay	DNN-based RL	Ours
Binary	NSL-KDD	0.7438	0.7370	0.7384	0.7629
	UNSW-NB15	0.6766	0.6469	0.5506	0.6834
	IoTID20	0.8553	0.9145	0.9014	0.9581
Multi-class	NSL-KDD	0.4481	0.6971	0.3308	0.7170
	UNSW-NB15	0.2668	0.3830	-	0.6119
	IoTID20	0.5580	0.3303	0.5581	0.7710

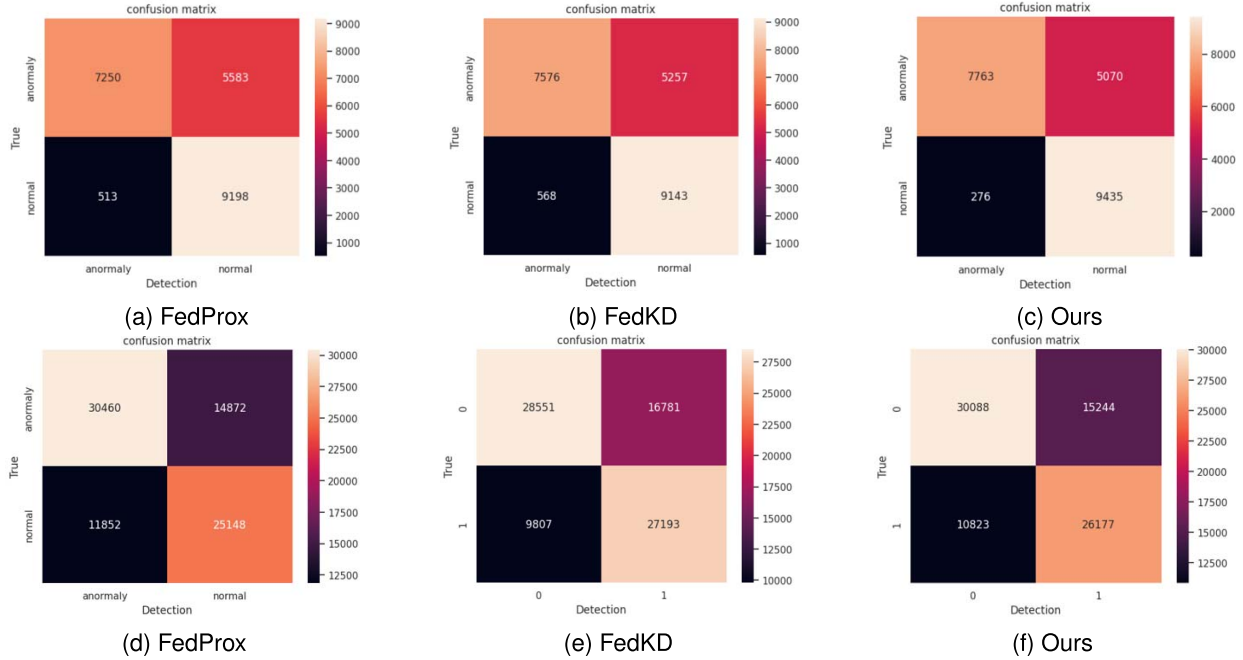


Fig. 10. Confusion matrix for binary classification using the test dataset of NSL-KDD and UNSW-NB15 dataset, respectively.

four methods. Specifically, compared with the one without FS, taking multi-classification as an example, the accuracy obtained by the one without FS is 0.4481, 0.2668, and 0.5580 for NSL-KDD, UNSW-NB15 and IoTID20 test dataset, respectively, while the accuracy gained by the proposed method is separately 0.7170, 0.6119 and 0.7710. The reason for this phenomenon is that FS is helpful in increasing the accuracy [15]. Compared with the one without LR decay, for instance, for IoTID20 dataset, the accuracy achieved by the proposed method is $\frac{0.9581}{0.9145} = 1.0477 \times$ greater than that of the one without LR decay in binary classification, while that in multi-class classification is $\frac{0.7710}{0.3303} = 2.3342 \times$. As mentioned in [82], large LR will cause the degradation of the model performance, while small LR will lead to slow convergence for model training. Since LR decay is adopted by the proposed method ($LR = LR \times 0.97^{q-1}$), the LR will be variable in each round, which can make the LR in the proposed method more suitable for the considered datasets. When comparing with the one adopts DNN-based representation layers (RLs), for binary classification, it can be seen that the accuracy

achieved by the proposed method on NSL-KDD, UNSW-NB15 and IoTID20 datasets are separately $\frac{0.7629-0.7384}{0.7384} \times 100\% = 3.3180\%$, $\frac{0.6834-0.5506}{0.5506} \times 100\% = 24.1191\%$ and $\frac{0.9581-0.9014}{0.9014} \times 100\% = 6.2902\%$ higher than those obtained by the DNN-based E-FPKD approach. In terms of multi-classification, for both NSL-KDD and IoTID20 datasets, it can be seen that the proposed method can acquire higher accuracy. As for the UNSW-NB15 dataset, when leveraging DNN as the representation layer, the achieved accuracy is very low, thereby we omit to mark the corresponding value in the Table. This phenomenon occurs because the convolutional layer is adopted by the proposed method, which can increase the network nonlinearity such that the accuracy can be increased [84].

To reduce the occupied space in this work, and for convenience, we show the confusion matrix (CM) for binary classification by using the NSL-KDD and UNSW-NB15 datasets only (as shown in Fig. 10), while we show CM for multi-class classification with IoTID20 dataset (shown as Fig. 11). According to Fig. 10 and Section VII-B, we derive Table IX. From this Table, compared with FedProx

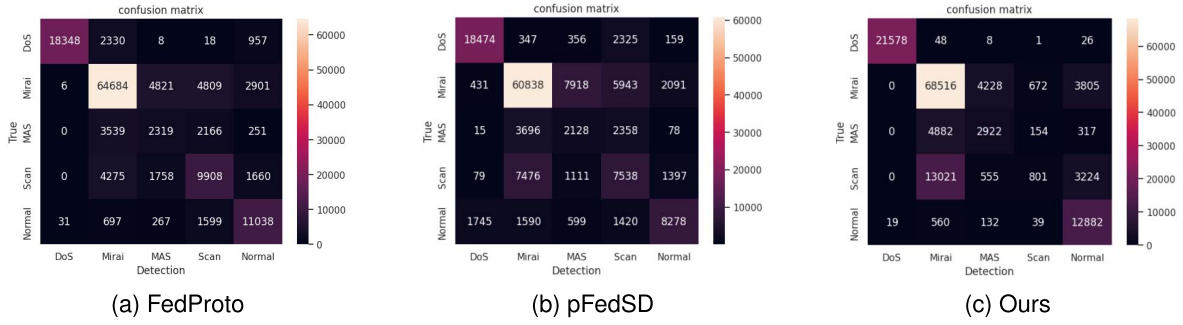


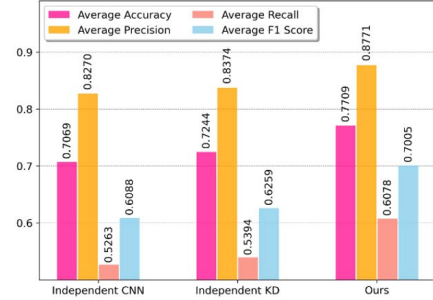
Fig. 11. Confusion matrix for multi-class classification using the test dataset of IoTID20 dataset.

TABLE IX
VARIOUS METRICS COMPARISON AMONG FEDPROX, FEDKD, AND THE
PROPOSED METHOD IN BINARY CLASSIFICATION

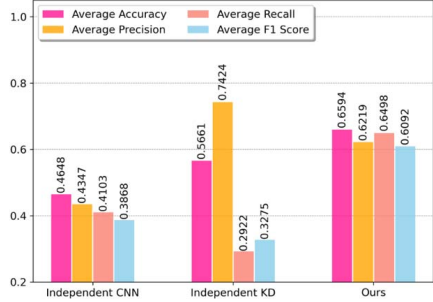
Dataset	Metric	FedProx	FedKD	Ours
NSL-KDD	Accuracy	0.7296	0.7416	0.7629
	FAR	0.0528	0.0585	0.0284
	Precision	0.9339	0.9303	0.9657
	Recall	0.5649	0.5904	0.6049
	F1 Score	0.7040	0.7223	0.7439
	AA	0.7374	0.7484	0.7709
UNSW-NB15	Accuracy	0.6754	0.6771	0.6834
	FAR	0.3203	0.2651	0.2925
	Precision	0.7199	0.7443	0.7355
	Recall	0.6719	0.6298	0.6637
	F1 Score	0.6951	0.6823	0.6977
	AA	0.6490	0.6564	0.6594

and FedKD, for NSL-KDD dataset, it can be seen that the accuracy, precision, recall, F1 score and AA achieved by the proposed method are the highest, while the achieved FAR is the lowest. As for UNSW-NB15 dataset, although the FAR achieved by the proposed method is not the smallest and the achieved precision and recall are not the largest, the achieved accuracy, F1 Score and AA are the largest. For multi-class classification, according to Fig. 11, the accuracy achieved by FedProto, pFedSD and the proposed method can be calculated as 0.7681, 0.7028, and 0.7710, respectively. The reasons for such phenomenon are as follows: 1) compared with FedProx, FedKD and pFedSD, only the proposed E-FPKD approach adds the regularization term (eq. (13)) to each local loss function. As stated in [32], the regularization term can help to gain a better representation net. 2) Regarding Fedproto, although it also uses the regularization term, KD is not adopted. Integrating KD into FL is an effective manner to handle data heterogeneity [17]. Thus, owing to KD, the performance of the proposed method in handling non-IID data is improved. Accordingly, we can infer that the proposed method is an effective way to detect cyber-attacks.

Next, we compare our proposed method with Independent CNN (ICNN) and Independent KD (IKD) approach regarding average accuracy, average precision, average recall, and average F1 score by using NSL-KDD and UNSW-NB15 datasets, as shown in Fig. 12. Notably, for ICNN and IKD approaches, CNN and KD are independently applied to each client. In addition, compared with CNN, the difference of KD is that KD has one more teacher network than CNN. Through Fig. 12, the



(a) NSL-KDD



(b) UNSW-NB15

Fig. 12. Comparison among CNN, KD and the proposed method using the test dataset of NSL-KDD and UNSW-NB15 dataset.

following contents can be observed: 1) for NSL-KDD dataset, four metrics achieved by the proposed method are separately 0.7709, 0.8771, 0.6078, and 0.7005, which are the highest among those three methods, followed by IKD approach and ICNN approach. 2) For the UNSW-NB15 dataset, we can observe that the proposed method can realize the highest AA, AR, and AFS, however, the achieved AP is the second-largest. The reason for this phenomenon is: some participating clients may only have a small amount of data, as per [66], lack of data is capable of affecting the performance. Regarding the proposed method, benefiting from the prototype aggregation process, the performance of handling heterogeneous data can be improved such that comparable results can be gained.

To further reveal the goodness of the proposed E-FPKD approach, we conduct the comparison with other methods over three datasets, as shown in Table X. In particular, FedGKD (2) [80], FedGKD (5) [80], and pFedSD [25] are KD-based FL approaches. Notably, (2) and (5) separately represent two and five latest generated global models (including the global

TABLE X
ACCURACY COMPARISON AMONG VARIOUS METHODS IN BOTH BINARY
AND MULTI-CLASS CLASSIFICATION

Dataset	Method	Accuracy Classification	
		Binary	Multi-class
NSL-KDD	FedAvg	0.6899	0.6363
	FedProto	0.7537	0.6856
	FedAU	0.7337	0.6433
	FedExp	0.7076	0.6112
	FedGKD (2)	0.6462	0.5946
	FedGKD (5)	0.6769	0.6629
	pFedSD	0.7451	0.7078
	Ours	0.7629	0.7170
UNSW-NB15	FedAvg	0.5645	0.4904
	FedProto	0.6629	0.5358
	FedAU	0.6537	0.6065
	FedExp	0.6238	0.3943
	FedGKD (2)	0.5732	0.4268
	FedGKD (5)	0.6539	0.4487
	pFedSD	0.6757	0.5565
	Ours	0.6834	0.6119
IoTID20	FedAvg	0.9015	0.6706
	FedProto	0.9151	0.7681
	FedAU	0.9445	0.7382
	FedExp	0.8253	0.4715
	FedGKD (2)	0.8715	0.4916
	FedGKD (5)	0.8953	0.7080
	pFedSD	0.9479	0.7028
	Ours	0.9581	0.7710

model generated in the current round) that will be averaged to produce the ensemble model. Through this Table, we can see the accuracy achieved by the proposed method in both binary and multi-class classification are the highest for the considered three datasets. Particularly, considering the IoTID20 dataset and binary classification as the example, the accuracy achieved by the proposed method is $\frac{0.9581}{0.9015} = 1.0628\times$, $1.0470\times$, $1.0144\times$, $1.1609\times$, $1.0994\times$, $1.0701\times$, and $1.0108\times$ greater than FedAvg [48], FedProto [18], FedAU [44], FedExp [45], FedGKD (2) [80], FedGKD (5) [80] and pFedSD [25], respectively. Accordingly, we can infer the proposed method can realize the most significant effect in terms of accuracy among all the methods shown in this Table. The reasons for obtaining those results are that: 1) compared with FedProto, we additionally integrate KD, where as mentioned previously, adding KD to FL is helpful for handling the non-IID data. 2) In contrast with other methods, a regularization term is added to the proposed approach, which can help to learn a better representation net. Consequently, the proposed method can gain the highest accuracy.

Robustness of the Proposed Method. To evaluate the robustness of the proposed method, same as [80], we also consider demonstrating the achieved accuracy across different rounds (as shown in Table XI). In this Table, compared with FedAvg and pFedSD, for NSL-KDD and IoTID20 datasets, the accuracy obtained by the proposed method is the best in both binary and multi-class classification at various rounds. As for UNSW-NB15 dataset, the proposed method can realize the maximum accuracy in binary classification. In multi-class classification, in comparison with FedAvg, the achieved accuracy via the proposed method is higher. However, compared with

pFedSD, the accuracy obtained via the proposed approach is lower when considering 5 rounds. This is because the model training is still in the exploration stage in the initial rounds, it is possible to get lower accuracy via the proposed method than that obtained by pFedSD for multi-class classification of the UNSW-NB15 dataset. This trend is changed when the round is increased to 10, 20 and 50. Hence, overall, it can be conjectured that the robustness of the proposed method is significant to some extent, compared to FedAvg and pFedSD.

In Table XII, we clarify the ODC (eq. (5)) obtained by various FL-based approach. From this Table, it can be known that the proposed method can get maximum ODC for the adopted three datasets among all the listed FL-based approaches. To be specific, the ODC obtained via the proposed method is separately 17198, 56265 and 132596 for NSL-KDD, UNSW-NB15, and IoTID20 datasets (test part) in binary classification, while the obtained ODC is 16343, 50380 and 106699, respectively, for multi-class classification. The reason for getting these results is that the proposed method can achieve the highest accuracy compared with other baselines (the achieved accuracy can be seen in Table X). The higher the accuracy, the higher the ODC. Thus, we can infer that the proposed method outperforms other listed FL-based methods.

C. Evaluation via Qualitative Analysis

1) *Qualitative Analysis by Analyzing Several Advantages:* Identical to [101] and [102] which made the qualitative comparison by listing several advantages, in this work, the same manner is adopted. As can be seen in Table XIII, we compare the proposed method with FedAvg, pFedSD, FedProto and FedGKD. Specifically, since FL can handle the data heterogeneity issue and all the considered methods are FL-based approaches, we can know all the methods in Table XIII can address NT data heterogeneity issues. Particularly, because the proposed method can achieve the maximum ODC (i.e., (5)) and the highest accuracy (as shown in Section VII-B2), it can be inferred that the performance of the proposed method in addressing the non-IID NT data of this work is the best among those methods. In addition, all of them can solve the data privacy issue caused by centralized gathering, since centralized data collection is not required. Considering the input data can be deduced through the given model [67], data leakage is taken into account. Among the considered methods, only FedProto and the proposed method can reduce data leakage risk caused by model inverse inference. This is because prototype aggregation is employed in each round for FedProto, while model aggregation (MA) is only adopted in the final round for the proposed method. Through such a manner, the model inverse inference can be mitigated. Apart from this, for future application, when a new client joins, only FedProto requires new client to conduct a fine-tuning process. The reason is that MA is adopted by FedAvg, pFedSD, and FedGKD in each round, and employed by the proposed method in the final round. Thereby, the final generated global model can be directly used by the newly joined client without the need of fine-tuning. As for FedProto, it generates global prototypes which can not be directly used

TABLE XI
EVALUATION OF THREE METHODS OVER THE TEST PART OF NSL-KDD, UNSW-NB15, AND IoTID20 AT DIFFERENT ROUNDS

Dataset	Method	# Round = 10		# Round = 20		# Round = 50		# Round = 100	
		Binary	Multi-class	Binary	Multi-class	Binary	Multi-class	Binary	Multi-class
NSL-KDD	FedAvg	0.6597	0.6098	0.6710	0.6183	0.6826	0.6173	0.6899	0.6363
	pFedSD	0.7061	0.6685	0.7105	0.6761	0.7157	0.6969	0.7451	0.7078
	Ours	0.7105	0.6833	0.7121	0.6970	0.7168	0.7037	0.7629	0.7170
Dataset	Method	# Round = 5		# Round = 10		# Round = 20		# Round = 50	
		Binary	Multi-class	Binary	Multi-class	Binary	Multi-class	Binary	Multi-class
UNSW-NB15	FedAvg	0.5193	0.3402	0.5489	0.4110	0.5505	0.4159	0.5645	0.4904
	pFedSD	0.4350	0.5072	0.4579	0.5343	0.6307	0.5437	0.6757	0.5565
	Ours	0.5506	0.4488	0.5973	0.5390	0.6821	0.5701	0.6834	0.6119
Dataset	Method	# Round = 5		# Round = 10		# Round = 15		# Round = 20	
		Binary	Multi-class	Binary	Multi-class	Binary	Multi-class	Binary	Multi-class
IoTID20	FedAvg	0.9004	0.5580	0.9009	0.6053	0.9015	0.6314	0.9172	0.6706
	pFedSD	0.9259	0.3955	0.9317	0.6112	0.9364	0.6676	0.9479	0.7028
	Ours	0.9459	0.7154	0.9512	0.7617	0.9529	0.7484	0.9581	0.7710

TABLE XII
ODC ACHIEVED BY EQ. (5) REALIZED BY VARIOUS METHODS ON THREE TEST DATASETS

Classification	Method	NSL-KDD		UNSW-NB15		IoTID20	
		Ground Truth	ODC	Ground Truth	ODC	Ground Truth	ODC
Binary	FedAvg	22544	15553	82332	46478	138390	126929
	FedProto		16849		54578		126634
	FedGKD(2)		14568		47190		120610
	FedGKD(5)		15260		53837		123896
	pFedSD		16798		55632		131178
	FedAU		16541		53824		130706
	FedExP		15953		51362		114209
	Ours		17198		56265		132596
Multi-class	FedAvg	22544	14345	82332	40375	138390	92803
	FedProto		15456		44111		106297
	FedGKD(2)		13404		35138		68031
	FedGKD(5)		14944		36942		97983
	pFedSD		15956		45814		97256
	FedAU		14502		49935		102156
	FedExP		13778		32464		65256
	Ours		16343		50380		106699

as a model. Further, only FedGKD requires the server buffer to save historical global models. Because FedGKD needs to save several historical global models for generating the ensemble model.

2) *Qualitative Analysis by Analyzing Computing Resource Usage:* In this part, we will compare the computing resource usage of the proposed method with FedAvg, pFedSD, FedProto, and FedGKD. In particular, we compare the server and each client of the proposed method with those baselines in terms of computing resource usage. Notably, after detecting the cyber-attacks with each method, we simply adopt a basic

rule-based method to directly block the malicious network traffic. Thus, for simplicity, we only consider the computing resource utilization for cyber-attack detection. Specifically, compared with FedAvg and FedProto, for each client, pre-training TNet is required by the proposed method, which causes the proposed method to consume additional computing resources. During the client's student network training, FedProto needs to compute the regularization term and the local prototypes, but the client of FedAvg does not need to. In contrast to FedProto, the proposed method also needs to calculate the KD loss. Accordingly, the client of proposed

TABLE XIII
QUALITATIVE COMPARISON AMONG VARIOUS METHODS REGARDING ADVANTAGES

Advantages	FedAvg	pFedSD	FedProto	FedGKD	Ours
Support NT data heterogeneity issue (i.e., non-IID NT data)	Yes	Yes	Yes	Yes	Yes & Best
Solve data privacy issue caused by centralized gathering	Yes	Yes	Yes	Yes	Yes
Reduce data leakage caused by model inverse inference	No	No	Yes	No	Yes
Eliminate the fine-tuning process for newly joined client	Yes	Yes	No	Yes	Yes
Need server buffer to save historical global models	No	No	No	Yes	No

method requires the most computational resources compared with FedAvg and FedProto. Regarding the server, model aggregation (MA) is adopted by FedAvg in each round, while prototype aggregation (PA) is employed by FedProto. When it comes to the proposed method, the PA is performed in each round, and the MA is only executed in the final round. Hence, we can infer the computing resource usage of the server of the proposed method is in the middle of FedAvg and FedProto. As for FedGKD and pFedSD, compared with FedAvg, those methods need to additionally compute the KD loss. In addition to KD loss, as the proposed method also needs to calculate the prototype in each client, the client computation resource utilization of the proposed method is higher than FedGKD's client and pFedSD's client. Moreover, both FedGKD and pFedSD adopt model aggregation in each round. For FedGKD, it is necessary to calculate the ensemble model by using several models. As such, among FedGKD, pFedSD and the proposed method, the server computing resource usage of the proposed method is the smallest, and that of FedGKD is the largest. Accordingly, the order of server's computing resource usage is: FedGKD > pFedSD = FedAvg > E-FPKD (Ours) > FedProto. Considering the purpose of this work is to maximize the ODC (5), for the proposed method, although the computing resources separately used by each client and the server are not the smallest, its efficiency (as shown in Section VII-B2) is the best in achieving (5). Therefore, it is feasible to adopt the proposed E-FPKD for cyber-attack detection.

VIII. DISCUSSION

Limitations. In this work, although the ODC achieved by the proposed solution is competitive compared to other baselines (e.g., FedProto), the proposed solution still has two limitations. Concretely, supervised learning is adopted to construct the proposed E-FPKD approach. Owing to this, to generate the model, labeled data are required. In such a way, unknown patterns will fail to be recognized [103]. In addition, in the final round, model aggregation is adopted to generate the global model. According to [67], with the given model, input data can be inferred. Thus, although the proposed solution can solve the data privacy issue caused by centralized gathering data, the risk of data leakage may still occur due to the potential of model inverse inference in the final round.

Future Potential Research Areas. Future potential research areas can be conducted based on above two limitations. Specifically, rather than supervised learning, unsupervised learning [103] (UL) or semi-supervised learning [104] (SSL) can be leveraged. Thus, combining FL with advanced UL or SSL can be a potential research direction. To mitigate the data leakage issue, model protecting technique

TABLE XIV
APPLICATION IN OTHER FIELDS VIA PROPOSED E-FPKD APPROACH
($\delta = 0.9$, # CLIENTS = 10)

Dataset	Accuracy
Smart Grid Stability (Binary)	0.8571
TD_SGE (Binary)	0.6626
TD_SGE (Multi-class)	0.5856

can be considered. For instance, masks and homomorphic encryption are possible to be employed since they can prevent data from being inferred by a variety of attacks (e.g., model inversion attack) [105].

IX. CONCLUSION

In this paper, cyber-attack prevention for prosumer-based EV charging stations is focused, where the network traffic data of all prosumers has heterogeneous characteristics (i.e., non-IID). In particular, the considered cyber-attack prevention process covers two aspects: cyber-attack detection and intervention. The purpose is to maximize the ODC. To tackle this aspect, an edge-assisted federated prototype knowledge distillation (E-FPKD) approach has been proposed. To improve accuracy, we have conducted feature selection with Pearson Correlation Coefficient before using the proposed E-FPKD approach. In addition, we have considered deploying each client model on a dedicated local edge server, where each client can launch its availability for participating in the FL process. Apart from this, we have integrated the KD technique for each client, which allows the student model can learn from the teacher model to gain useful information. Besides, we have combined the KD with the prototype aggregation for performance enhancement. Then, the global model obtained by averaging all the participating clients' personal models in the final round is adopted for the application stage such that the newly joined clients can directly use it without the necessity of fine-tuning. Finally, to intervene in the detected cyber attacks, a rule-based method called “if... then” is employed. The conducted experiments clarify the practicality of the proposed method. For instance, in binary classification, the proposed method can achieve the highest ODC for the considered three datasets separately with a value of 17198, 56265, and 132596, compared with other considered FL-based methods.

APPENDIX APPLICATIONS IN OTHER DOMAINS

Applicability. In this part, we shed light on the *applicability* of the proposed E-FPKD approach. As stated before, 1D-CNN combined with FC network, and supervised learning

are used to construct the proposed method. Thus, the application range of the proposed method can be: since 1D-CNN is utilized to receive the input data, if the data is labeled data and each data record can be denoted as a vector, the proposed method can be leveraged. Next, as examples, we apply the proposed approach to two cases in other domains, where the detailed scenario will be discussed based on each case.

Case 1: Smart Grid Stability Analysis. Considering an urban area that consists of multiple regions and a distribution system operator (DSO) for buying power grid energy, particularly, each region is formed by one energy producer and multiple energy consumers. For such an urban area, we aim to analyze its smart grid stability by the proposed approach. Specifically, each region is considered as the client of the proposed approach and the aggregation process will be executed in the DSO. To perform this, Smart Grid Stability (SGS) dataset [106] is adopted since each record of that dataset is labeled by stable or unstable, and it contains the information regarding a energy producer and multiple energy consumers. As SGS dataset only includes two categories (i.e., {unstable, stable}), only binary classification scenario is considered in this case. Further, we use *Dirichlet Distribution* with parameter $\delta = 0.9$ to prepare data for 10 clients.

Case 2: Energy Theft Detection. Consider a smart grid environment that is composed of multiple energy users and a DSO (same as case 1), we aim to detect the energy theft towards such an environment with the proposed approach. Specifically, each energy user is considered as each client, and the aggregation process is performed in DSO. For evaluation, we employ Theft detection in smart grid environment dataset (TD_SGE) [107] which contains energy consumption data belonging to *normal* or *six types of thefts*. Thus, we consider both binary (i.e., {Theft, Normal}) and multi-class classification (i.e., {Theft1, Theft2, Theft3, Theft4, Theft5, Theft6, Normal}) scenarios for TD_SGE dataset. In addition, we also use *Dirichlet Distribution* with parameter $\delta = 0.9$ to split data for 10 clients.

In Table XIV, we clarify the accuracy achieved by the proposed method for both Case 1 and Case 2. We can see that the proposed approach can also be applied to other domains.

REFERENCES

- [1] L. Zou, M. S. Munir, Y. K. Tun, S. Kang, and C. S. Hong, "Intelligent EV charging for urban prosumer communities: An auction and multi-agent deep reinforcement learning approach," *IEEE Trans. Netw. Service Manag.*, vol. 19, no. 4, pp. 4384–4407, Dec. 2022.
- [2] S. Islam, S. Badsha, S. Sengupta, I. Khalil, and M. Atiquzzaman, "An intelligent privacy preservation scheme for EV charging infrastructure," *IEEE Trans. Ind. Informat.*, vol. 19, no. 2, pp. 1238–1247, Feb. 2023.
- [3] T. Morstyn, C. Crozier, M. Deakin, and M. D. McCulloch, "Conic optimization for electric vehicle station smart charging with battery voltage constraints" *IEEE Trans. Transport. Electrification*, vol. 6, no. 2, pp. 478–487, Jun. 2020.
- [4] L. Zou, C. M. Thwal, S.-B. Park, and C. S. Hong, "Edge-assisted attention-based federated learning for multi-step EVSE-enabled prosumer energy demand prediction," in *Proc. Int. Conf. Inf. Netw. (ICOIN)*, Bangkok, Thailand, Jan. 2023, pp. 116–121.
- [5] A. Takiddin, M. Ismail, and E. Serpedin, "Robust detection of electricity theft against evasion attacks in smart grids," in *Proc. IEEE Int. Conf. Commun.*, Montreal, QC, Canada, 2021, pp. 1–6.
- [6] S. Yip, W. Tan, C. Tan, M. Gan, and K. Wong, "An anomaly detection framework for identifying energy theft and defective meters in smart grids," *Int. J. Electr. Power Energy Syst.*, vol. 101, pp. 189–203, Oct. 2018.
- [7] K. Zheng, Y. Wang, Q. Chen, and Y. Li, "Electricity theft detecting based on density-clustering method," in *Proc. IEEE Innov. Smart Grid Technol. Asia (ISGT-Asia)*, Auckland, New Zealand, Dec. 2017, pp. 1–6.
- [8] L. Zou, S. Park, S. M. Kang, and C. S. Hong, "A multiple cloud-aided federated learning-based security module towards EVCS energy management system," in *Proc. Korea Softw. Congr.*, Dec. 2022, pp. 289–291.
- [9] Y. Yue, X. Chen, Z. Han, X. Zeng, and Y. Zhu, "Contrastive learning enhanced intrusion detection," *IEEE Trans. Netw. Service Manag.*, vol. 19, no. 4, pp. 4232–4247, Dec. 2022.
- [10] Y. M. Saputra, D. T. Hoang, D. N. Nguyen, E. Dutkiewicz, M. D. Mueck, and S. Srikanteswara, "Energy demand prediction with federated learning for electric vehicle networks," in *Proc. IEEE Global Commun. Conf. (GLOBECOM)*, Waikoloa, HI, USA, Dec. 2019, pp. 1–6.
- [11] S. Abdulrahman, H. Tout, H. Ould-Slimane, A. Mourad, C. Talhi, and M. Guizani, "A survey on federated learning: The journey from centralized to distributed on-site learning and beyond," *IEEE Internet Things J.*, vol. 8, no. 7, pp. 5476–5497, Apr. 2021.
- [12] Y. Kim, E. A. Hakim, J. Haraldson, H. Eriksson, J. M. B. da Silva, and C. Fischione, "Dynamic clustering in federated learning," in *Proc. IEEE Int. Conf. Commun.*, Montreal, QC, Canada, Jun. 2021, pp. 1–6.
- [13] T. Huang, W. Lin, W. Wu, L. He, K. Li, and A. Y. Zomaya, "An efficiency-boosting client selection scheme for federated learning with fairness guarantee," *IEEE Trans. Parallel Distrib. Syst.*, vol. 32, no. 7, pp. 1552–1564, Jul. 2021.
- [14] M. Habiba, M. R. Islam, S. Mueen, and A. S. Ali, "Edge intelligence for network intrusion prevention in IoT ecosystem," *Comput. Electr. Eng.*, vol. 108, May 2023, Art. no. 108727.
- [15] T. Wisanwanichthan and M. Thammawichai, "A double-layered hybrid approach for network intrusion detection system using combined naive bayes and SVM," *IEEE Access*, vol. 9, pp. 138432–138450, 2021.
- [16] F. Tung and G. Mori, "Similarity-preserving knowledge distillation," in *Proc. IEEE/CVF Int. Conf. Comput. Vis. (ICCV)*, 2019, pp. 1365–1374.
- [17] J. Tang et al., "FedRAD: Heterogeneous federated learning via relational adaptive distillation," *Sensors*, vol. 23, no. 14, p. 6518, Jul. 2023.
- [18] Y. Tan et al., "FedProto: Federated Prototype Learning across Heterogeneous Clients," in *Proc. AAAI Conf. Artif. Intell.*, vol. 36, Jun. 2022, pp. 8432–8440.
- [19] M. ElKashlan, H. Aslan, M. S. Elsayed, A. D. Jurcut, and M. A. Azer, "Intrusion detection for electric vehicle charging systems (EVCS)," *Algorithms*, vol. 16, no. 2, p. 75, Jan. 2023.
- [20] M. ElKashlan, M. S. Elsayed, A. D. Jurcut, and M. Azer, "A machine learning-based intrusion detection system for IoT electric vehicle charging stations (EVCSs)," *Electronics*, vol. 12, no. 4, p. 1044, Feb. 2023.
- [21] M. Basnet and M. H. Ali, "Deep learning-based intrusion detection system for electric vehicle charging station," in *Proc. 2nd Int. Conf. Smart Power Internet Energy Syst. (SPIES)*, Bangkok, Thailand, Sep. 2020, pp. 408–413.
- [22] M. A. Rahman, Y. Al-Saggaf, and T. Zia, "A data mining framework to predict cyber attack for cyber security," in *Proc. 15th IEEE Conf. Ind. Electron. Appl. (ICIEA)*, Kristiansand, Norway, Nov. 2020, pp. 207–212.
- [23] V. V. Vegesna, "Utilising VAPT technologies (vulnerability assessment & penetration testing) as a method for actively preventing cyberattacks," *Int. J. Manage. Technol. Eng.*, vol. 12, p. 14, Jul. 2022.
- [24] P. Qi, X. Zhou, Y. Ding, Z. Zhang, S. Zheng, and Z. Li, "FedBKD: Heterogeneous federated learning via bidirectional knowledge distillation for modulation classification in IoT-edge system," *IEEE J. Sel. Topics Signal Process.*, vol. 17, no. 1, pp. 189–204, Jan. 2023.
- [25] H. Jin et al., "Personalized edge intelligence via federated self-knowledge distillation," *IEEE Trans. Parallel Distrib. Syst.*, vol. 34, no. 2, pp. 567–580, Feb. 2023.
- [26] H. Q. Le, L. X. Nguyen, S.-B. Park, and C. S. Hong, "Layer-wise knowledge distillation for cross-device federated learning," in *Proc. Int. Conf. Inf. Netw. (ICOIN)*, Bangkok, Thailand, Jan. 2023, pp. 526–529.
- [27] C. Wu, F. Wu, L. Lyu, Y. Huang, and X. Xie, "Communication-efficient federated learning via knowledge distillation," *Nat. Commun.*, vol. 13, p. 2032, Apr. 2022.

- [28] T. Gao, X. Jin, and Y. Lai, "Model heterogeneous federated learning for intrusion detection based on knowledge distillation," in *Proc. 12th Int. Conf. Commun. Netw. Security*, Beijing, China, Dec. 2022, pp. 30–35.
- [29] R. Zhao, L. Yang, Y. Wang, Z. Xue, G. Gui, and T. Ohtsuki, "A semi-supervised federated learning scheme via knowledge distillation for intrusion detection," in *Proc. IEEE Int. Conf. Commun.*, Seoul, Republic of Korea, May 2022, pp. 2688–2693.
- [30] J. Shen, W. Yang, Z. Chu, J. Fan, D. Niyato, and K. Lam, "Effective intrusion detection in heterogeneous Internet-of-Things networks via ensemble knowledge distillation-based federated learning," Jan. 2024, *arXiv:2401.11968*.
- [31] L. Zou, H. Q. Le, A. D. Raha, D. U. Kim, and C. S. Hong, "EFCKD: edge-assisted federated contrastive knowledge distillation approach for energy management: Energy theft perspective," in *Proc. 24th Asia-Pacific Netw. Oper. Manage. Symp. (APNOMS)*, Sejong, Republic of Korea, Sep. 2023, pp. 30–35.
- [32] D. Cheng, L. Zhang, C. Bu, X. Wang, H. Wu, and A. Song, "ProtoHAR: Prototype guided personalized federated learning for human activity recognition," *IEEE J. Biomed. Health Inform.*, vol. 27, no. 8, pp. 3900–3911, Aug. 2023.
- [33] A. Wang, L. Yang, H. Wu, and Y. Iwahori, "Heterogeneous defect prediction based on federated prototype learning," *IEEE Access*, vol. 11, pp. 98618–98632, 2023.
- [34] T. Gao, X. Liu, Y. Yang, and G. Wang, "FEDMBP: Multi-branch prototype federated learning on heterogeneous data," in *Proc. IEEE Int. Conf. Image Process. (ICIP)*, Kuala Lumpur, Malaysia, Oct. 2023, pp. 2180–2184.
- [35] B. Li, W. Gao, J. Xie, M. Gong, L. Wang, and H. Li, "Prototype-based decentralized federated learning for the heterogeneous time-varying IoT systems," *IEEE Internet Things J.*, vol. 11, no. 4, pp. 6916–6927, Feb. 2024, doi: [10.1109/JIOT.2023.3313118](https://doi.org/10.1109/JIOT.2023.3313118).
- [36] S. Wang, K. Yan, S. Wang, and C. Liu, "An integrated intrusion detection system for cluster-based wireless sensor networks," *Expert Syst. Appl.*, vol. 38, pp. 15234–15243, Nov.–Dec. 2011.
- [37] Jun. 1, 2023, "NSL-KDD dataset," University of New Brunswick, [Online]. Available: <https://www.unb.ca/cic/datasets/nsll.html>
- [38] N. Moustafa and J. Slay, "UNSW-NB15: A comprehensive data set for network intrusion detection systems (UNSW-NB15 network data set)," in *Proc. Mil. Commun. Inf. Syst. Conf. (MilCIS)*, Canberra, ACT, Australia, Nov. 2015, pp. 1–6.
- [39] N. Moustafa and J. Slay, "The evaluation of network anomaly detection systems: Statistical analysis of the UNSW-NB15 data set and the comparison with the KDD99 data set," *Inf. Security J. Global Perspect.*, vol. 25, pp. 18–31, Apr. 2016.
- [40] N. Moustafa, J. Slay, and G. Creech, "Novel geometric area analysis technique for anomaly detection using trapezoidal area estimation on large-scale networks," *IEEE Trans. Big Data*, vol. 5, no. 4, pp. 481–494, Dec. 2019.
- [41] N. Moustafa, G. Creech, and J. Slay, "Big data analytics for intrusion detection system: Statistical decision-making using finite Dirichlet mixture models," in *Data Analytics and Decision Support for Cybersecurity*. Cham, Switzerland: Springer, Aug. 2017, pp. 127–156.
- [42] M. Sarhan, S. Layeghy, N. Moustafa, and M. Portmann, "NetFlow datasets for machine learning-based network intrusion detection systems," in *Proc. 10th EAI Int. Conf. Big Data Technol. Appl. 13th EAI Int. Conf. Wireless Internet*, Dec. 2020, pp. 117–135.
- [43] I. Ullah and Q. H. Mahmoud, "A scheme for generating a dataset for anomalous activity detection in IoT networks," in *Advances in Artificial Intelligence*, C. Goutte and X. Zhu, Eds. Cham, Switzerland: Springer, May 2020, pp. 508–520.
- [44] S. Wang and M. Ji, "A lightweight method for tackling unknown participation statistics in federated averaging," in *Proc. ICLR*, Vienna, Austria, 2024, pp. 1–40.
- [45] D. Jhunjunwala, S. Wang, and G. Joshi, "FedExp: Speeding up federated averaging via extrapolation," in *Proc. ICLR*, May 2023, pp. 1–35.
- [46] S. Torquato, O. U. Uche, and F. H. Stillinger, "Random sequential addition of hard spheres in high Euclidean dimensions," *Phys. Rev. E, Stat. Phys. Plasmas Fluids Relat. Interdiscip. Top.*, vol. 74, Dec. 2006, Art. no. 61308.
- [47] G. D. Németh, M. Á. Lozano, N. Quadrianto, and N. Oliver, "A snapshot of the frontiers of client selection in federated learning," in *Proc. Trans. Mach. Learn. Res.*, Dec. 2022, pp. 1–25.
- [48] H. B. McMahan, E. Moore, D. Ramage, S. Hampson, and B. A. y Arcas, "Communication-efficient learning of deep networks from decentralized data," in *Proc. 20th Int. Conf. Artif. Intell. Stat.*, 2017, pp. 1273–1282.
- [49] J. Geiping, H. Bauermeister, H. Dröge, and M. Moeller, "Inverting Gradients—How easy is it to break privacy in federated learning?" in *Proc. 34th Int. Conf. Neural Inf. Process. Syst.*, Dec. 2020, pp. 16937–16947.
- [50] L. Zou, M. S. Munir, Y. K. Tun, S. S. Hassan, P. S. Aung, and C. S. Hong, "When hierarchical federated learning meets stochastic game: Toward an intelligent UAV charging in urban prosumers," *IEEE Internet Things J.*, vol. 10, no. 12, pp. 10438–10461, Jun. 2023.
- [51] L. Wang and K.-J. Yoon, "Knowledge distillation and student-teacher learning for visual intelligence: A review and new outlooks," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 44, no. 6, pp. 3048–3068, Jun. 2022.
- [52] B. Peng et al., "Correlation congruence for knowledge distillation," in *Proc. IEEE/CVF Int. Conf. Comput. Vis. (ICCV)*, 2019, pp. 5007–5016.
- [53] G. Hinton, O. Vinyals, and J. Dean, "Distilling the knowledge in a neural network," Mar. 2015, *arXiv:1503.02531*.
- [54] X. Shang, Y. Lu, Y.-M. Cheung, and H. Wang, "FEDIC: Federated learning on Non-IID and long-tailed data via calibrated distillation," in *Proc. IEEE Int. Conf. Multimedia and Expo (ICME)*, Taipei, Taiwan, Jul. 2022, pp. 1–6.
- [55] M. Nakashima, A. Sim, Y. Kim, J. Kim, and J. Kim, "Automated feature selection for anomaly detection in network traffic data," *ACM Trans. Manage. Inf. Syst.*, vol. 12, no. 18, pp. 1–28, Jun. 2021.
- [56] G. Li, A. Zhang, Q. Zhang, D. Wu, and C. Zhan, "Pearson correlation coefficient-based performance enhancement of broad learning system for stock price prediction," *IEEE Trans. Circuits Syst. II, Exp. Briefs*, vol. 69, no. 5, pp. 2413–2417, May 2022.
- [57] Y. Mu, X. Liu, and L. Wang, "A Pearson's correlation coefficient based decision tree and its parallel implementation," *Inf. Sci.*, vol. 435, pp. 40–58, Apr. 2018.
- [58] H. Zhou, Z. Deng, Y. Xia, and M. Fu, "A new sampling method in particle filter based on Pearson correlation coefficient," *Neurocomputing*, vol. 216, pp. 208–215, Dec. 2016.
- [59] P. Rungskunroch, Z. Shen, and S. Kaewunruen, "Benchmarking socio-economic impacts of high-speed rail networks using K-nearest neighbour and Pearson's correlation coefficient techniques through computational model-based analysis," *Appl. Sci.*, vol. 12, no. 3, p. 1520, Jan. 2022.
- [60] J. H. Cho and B. Hariharan, "On the efficacy of knowledge distillation," in *Proc. IEEE/CVF Int. Conf. Comput. Vis. (ICCV)*, 2019, pp. 4794–4802.
- [61] Y. Tian, D. Su, S. Lauria, and X. Liu, "Recent advances on loss functions in deep learning for computer vision," *Neurocomputing*, vol. 497, pp. 129–158, Aug. 2022.
- [62] J. D. L. Torre, D. Puig, and A. Valls, "Weighted kappa loss function for multi-class classification of ordinal data in deep learning," *Pattern Recognit. Lett.*, vol. 105, pp. 144–154, Apr. 2018.
- [63] H.-X. Gao, S. Kuenzel, and X.-Y. Zhang, "A hybrid ConvLSTM-based anomaly detection approach for combating energy theft," *IEEE Trans. Instrum. Meas.*, vol. 71, pp. 1–10, Aug. 2022.
- [64] Y. Wang, X. Ma, Z. Chen, Y. Luo, J. Yi, and J. Bailey, "Symmetric cross entropy for robust learning with noisy labels," in *Proc. IEEE/CVF Int. Conf. Comput. Vis. (ICCV)*, 2019, pp. 322–330.
- [65] E. Gordon-Rodriguez, G. Loaiza-Ganem, G. Pleiss, and J. P. Cunningham, "Uses and abuses of the cross-entropy loss: Case studies in modern deep learning," in *Proc. 'I Can't Believe It's Not Better!' NeurIPS Workshops*, vol. 137, Dec. 2020, pp. 1–10.
- [66] W. Yoon, C. H. So, J. Lee, and J. Kang, "CollaboNet: Collaboration of deep neural networks for biomedical named entity recognition," *BMC Bioinform.*, vol. 20, p. 249, May 2019.
- [67] Z. Xiong, Z. Cai, D. Takabi, and W. Li, "Privacy threat and defense for federated learning with non-i.i.d. data in AIoT," *IEEE Trans. Ind. Inform.*, vol. 18, no. 2, pp. 1310–1321, Feb. 2022.
- [68] P. J. Freire et al., "Performance versus complexity study of neural network equalizers in coherent optical systems," *J. Lightw. Technol.*, vol. 39, no. 19, pp. 6085–6096, Oct. 2021.
- [69] K. He and J. Sun, "Convolutional neural networks at constrained time cost," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2015, pp. 5353–5360.
- [70] C. Pan et al., "Towards fair graph federated learning via incentive mechanisms," Dec. 2023, *arXiv:2312.13306*.

- [71] X. Li, Z. Wu, W. Zhang, Y. Zhu, R. Li, and G. Wang, "FedGTA: Topology-aware averaging for federated graph learning," *Proc. VLDB Endow.*, vol. 17, pp. 41–50, Sep. 2023.
- [72] B. Salehi, J. Gu, D. Roy, and K. Chowdhury, "FLASH: Federated learning for automated selection of high-band mmWave sectors," in *Proc. IEEE Conf. Comput. Commun.*, London, U.K., May 2022, pp. 1719–1728.
- [73] Y. Shang, Z. Li, S. Li, Z. Shao, and L. Jian, "An information security solution for vehicle-to-grid scheduling by distributed edge computing and federated deep learning," *IEEE Trans. Ind. Appl.*, vol. 60, no. 3, pp. 4381–4395, May/Jun. 2024.
- [74] O. Jogunola, W. Wang, and B. Adebisi, "Prosumers matching and least-cost energy path optimisation for peer-to-peer energy trading," *IEEE Access*, vol. 8, pp. 95266–95277, 2020.
- [75] H. Li et al., "FedTP: Federated learning by transformer personalization," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 35, no. 10, pp. 13426–13440, Oct. 2024.
- [76] G. Yan, H. Wang, and J. Li, "Seizing critical learning periods in federated learning," in *Proc. AAAI Conf. Artif. Intell.*, Jun. 2022, pp. 1–9.
- [77] K. Kwon, H. Na, H. Lee, and N. S. Kim, "Adaptive knowledge distillation based on entropy," in *Proc. IEEE Int. Conf. Acoust., Speech Signal Process. (ICASSP)*, Barcelona, Spain, May 2020, pp. 7409–7413.
- [78] T. Liang, M. Wang, J. Chen, D. Chen, Z. Luo, and V. C. M. Leung, "Compressing the multiobject tracking model via knowledge distillation," *IEEE Trans. Comput. Social Syst.*, vol. 11, no. 2, pp. 2713–2723, Apr. 2024.
- [79] T. Li, A. K. Sahu, M. Zaheer, M. Sanjabi, A. Talwalkar, and V. Smith, "Federated optimization in heterogeneous networks," in *Proc. Mach. Learn. Syst. (MLSys)*, 2020, pp. 429–450.
- [80] D. Yao et al., "FedGKD: Toward heterogeneous federated learning via global knowledge distillation," *IEEE Trans. Comput.*, vol. 73, no. 1, pp. 3–17, Jan. 2024.
- [81] Y. Sun, S. Todorovic, and S. Goodison, "Local-learning-based feature selection for high-dimensional data analysis," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 32, no. 9, pp. 1610–1626, Sep. 2010.
- [82] Y. Zhang, T. Zhou, X. Huang, L. Cao, and Q. Zhou, "Fault diagnosis of rotating machinery based on recurrent neural networks," *Measurement*, vol. 171, Feb. 2021, Art. no. 108774.
- [83] W. Xu, Y. Zhang, and X. Tang, "Parallelizing DNN training on GPUs: Challenges and opportunities," in *Proc. Companion Proc. Web Conf.*, Jun. 2021, pp. 174–178.
- [84] R. Rahimilarki, Z. Gao, N. Jin, and A. Zhang, "Convolutional neural network fault classification based on time-series analysis for benchmark wind turbine machine," *Renew. Energy*, vol. 185, pp. 916–931, Feb. 2022.
- [85] M. Tavallaei, E. Bagheri, W. Lu, and A. A. Ghorbani, "A detailed analysis of the KDD CUP 99 data set," in *Proc. IEEE Symp. Comput. Intell. Security Defense Appl.*, Ottawa, ON, Canada, Jul. 2009, pp. 1–6.
- [86] A. Bhardwaj, V. Mangat, and R. Vig, "Hyperband tuned deep neural network with well posed stacked sparse autoencoder for detection of DDoS attacks in cloud," *IEEE Access*, vol. 8, pp. 181916–181929, 2020.
- [87] Y. Yang and P. Zhao, "Research on dung beetle optimization based stacked sparse autoencoder for network situation element extraction," *IEEE Access*, vol. 12, pp. 24014–24026, 2024.
- [88] I. Ullah and Q. H. Mahmoud, "A technique for generating a botnet dataset for anomalous activity detection in IoT networks," in *Proc. IEEE Int. Conf. Syst., Man, Cybern. (SMC)*, Toronto, ON, Canada, Oct. 2020, pp. 134–140.
- [89] A. A. Alsulami, Q. A. Al-Haija, A. Tayeb, and A. Alqahtani, "An intrusion detection and classification system for IoT traffic with improved data engineering," *Appl. Sci.*, vol. 12, p. 23, Dec. 2022, Art. no. 12336.
- [90] V. Surya and M. M. Selvam, "An effective machine learning approach for IoT intrusion detection system based on SMOTE," in *Proc. 6th Int. Conf. Electron., Commun. Aerosp. Technol.*, Coimbatore, India, Dec. 2022, pp. 905–911.
- [91] M. Omar and L. George, "Toward a lightweight machine learning based solution against cyber-intrusions for IoT," in *Proc. IEEE 46th Conf. Local Comput. Netw.*, Edmonton, AB, Canada, Oct. 2021, pp. 519–524.
- [92] O. Hoekstra, W. Hurst, and J. Tummers, "Healthcare related event prediction from textual data with machine learning: A systematic literature review," *Healthcare Anal.*, vol. 2, Nov. 2022, Art. no. 100107.
- [93] E. Fedorchenko, E. Novikova, and A. Shulepov, "Comparative review of the intrusion detection systems based on federated learning: Advantages and Open challenges," *Algorithms*, vol. 15, p. 247, Jul. 2022.
- [94] T. Alldieck, C. H. Bahnsen, and T. B. Moeslund, "Context-aware fusion of RGB and thermal imagery for traffic monitoring," *Sensors*, vol. 16, p. 1947, Nov. 2016.
- [95] Milan, H. Sardana, and K. Singh, "Reducing false alarms in intrusion detection systems—A survey," *Int. Res. J. Eng. Technol.*, vol. 5, pp. 9–12, Feb. 2018.
- [96] D. Jing and H.-B. Chen, "SVM based network intrusion detection for the UNSW-NB15 dataset," in *Proc. IEEE 13th Int. Conf. ASIC (ASICON)*, Chongqing, China, 2019, pp. 1–4.
- [97] J. Kim, N. Shin, S. Y. Jo, and S. H. Kim, "Method of intrusion detection using deep neural network," in *Proc. IEEE Int. Conf. Big Data Smart Comput. (BigComp)*, Feb. 2017, pp. 313–316.
- [98] T. Anwar and S. Zakir, "Deep learning based diagnosis of COVID-19 using chest CT-scan images," in *Proc. IEEE 23rd Int. Multitopic Conf. (INMIC)*, Bahawalpur, Pakistan, Nov. 2020, pp. 1–5.
- [99] J. Zhang, X. Yu, Y. Liu, J. Yu, and H. Gao, "MFCP-Net: Multiscale feature correlation perception network for image segmentation based on background prior," *IEEE Trans. Instrum. Meas.*, vol. 72, pp. 1–11, 2022.
- [100] M. Abdel-Basset, H. Hawash, K. M. Sallam, I. Elgendi, K. Munasinghe, and A. Jamalipour, "Efficient and lightweight convolutional networks for IoT malware detection: A federated learning approach," *IEEE Internet Things J.*, vol. 10, no. 8, pp. 7164–7173, Apr. 2023.
- [101] M. H. Ur Rehman, K. Salah, E. Damiani, and D. Svetinovic, "Towards blockchain-based reputation-aware federated learning," in *Proc. IEEE Conf. Comput. Commun. Workshops (INFOCOM WKSHPS)*, Toronto, ON, Canada, Jul. 2020, pp. 183–188.
- [102] S. M. A. Abbas, M. A. Khan, W. Boulila, A. Kouba, M. S. Khan, and J. Ahmad, "UAVs and blockchain synergy: Enabling secure reputation-based federated learning in smart cities," *IEEE Access*, vol. 12, pp. 154035–154053, 2024, doi: [10.1109/ACCESS.2024.3432610](https://doi.org/10.1109/ACCESS.2024.3432610).
- [103] M. Verkerken, L. D'hooge, T. Wauters, B. Volckaert, and F. De Turck, "Unsupervised machine learning techniques for network intrusion detection on modern data," in *Proc. 4th Cyber Security Netw. Conf. (CSNet)*, Lausanne, Switzerland, Oct. 2020, pp. 1–8.
- [104] Y. Gao, Y. Liu, Y. Jin, J. Chen, and H. Wu, "A novel semi-supervised learning approach for network intrusion detection on cloud-based robotic system," *IEEE Access*, vol. 6, pp. 50927–50938, 2018.
- [105] L. Zhang, J. Xu, P. Vijayakumar, P. K. Sharma, and U. Ghosh, "Homomorphic encryption-based privacy-preserving federated learning in IoT-enabled healthcare system," *IEEE Trans. Netw. Sci. Eng.*, vol. 10, no. 5, pp. 2864–2880, Sep./Oct. 2023.
- [106] "Smart grid stability." kaggle. Accessed: Apr. 2024. [Online]. Available: <https://www.kaggle.com/datasets/pcbreviglieri/smart-grid-stability>
- [107] S. Zidi, A. Mihoub, S. M. Qaisar, M. Krichen, and Q. A. Al-Haija, "Theft detection in smart grid environment." Mendeley Data. Jun. 2022. Accessed: Apr. 2024. [Online]. Available: <https://data.mendeley.com/datasets/c3c7329tjj/3>



Luyao Zou received the B.E. degree in electrical engineering and automation from NingboTech University (current name), China, in 2014, the M.S. degree in the major of computer science and engineering from the Department of Computer, Information and Communications Engineering, Konkuk University, South Korea, in 2018, and the Ph.D. degree in computer engineering from the Department of Computer Science and Engineering, Kyung Hee University, South Korea, in 2024, where she is currently working as a Postdoctoral Fellow.

Her current research interests include edge intelligence and deep learning.



Quang Hieu Vo received the B.S. degree in electrical engineering from the Ho Chi Minh City University of Technology, Ho Chi Minh, Vietnam, in 2017, and the Ph.D. degree in computer science and engineering from Kyung Hee University, South Korea, in 2024, where he was working on efficient deep learning architecture. From 2017 to 2018, he worked with eSilicon Vietnam Inc., Ho Chi Minh, as an ASIC Design Engineer. Since 2024, he has been a Research Engineer with Samsung Electric, South Korea. His research interests include the area

of computer vision, machine learning, VLSI system design, deep learning hardware architecture, and automated design optimization.



Chaoning Zhang (Senior Member, IEEE) received the B.E. and M.E. degrees in electrical engineering from the Harbin Institute of Technology, China, in 2012 and 2015, respectively, and the Ph.D. degree from KAIST in 2018. Since 2019, He has been an Assistant Professor with the Department of Artificial Intelligence, School of Computing, Kyung Hee University. Prior to that, he worked as a Postdoctoral Researcher with KAIST. His research interests include but are not limited to adversarial machine learning and self-supervised learning for

addressing model robustness and data efficiency issues in computer vision and beyond.



Kitae Kim (Graduate Student Member, IEEE) received the B.S., M.S., and Ph.D. degrees in computer science and engineering from Kyung Hee University, Seoul, South Korea, in 2017, 2019, and 2024, respectively, where he is currently a Postdoctoral Researcher with the Networking Intelligence Laboratory. His research focuses on 5G/6G wireless communication, AI/ML-based wireless networks, and channel estimation and prediction.



Choong Seon Hong (Fellow, IEEE) received the B.S. and M.S. degrees in electronic engineering from Kyung Hee University, Seoul, South Korea, in 1983 and 1985, respectively, and the Ph.D. degree from Keio University, Tokyo, Japan, in 1997. In 1988, he joined KT, Seongnam, Gyeonggi, South Korea, where he was involved in broadband networks as a member of the Technical Staff. Since 1993, he has been with Keio University. He was with the Telecommunications Network Laboratory, KT, as a Senior Member of Technical Staff and as the

Director of the Networking Research Team until 1999. Since 1999, he has been a Professor with the Department of Computer Science and Engineering, Kyung Hee University. His research interests include future Internet, intelligent edge computing, network management, and network security. He has served as the General Chair, the TPC Chair/Member, or an Organizing Committee Member of international conferences, such as the Network Operations and Management Symposium, International Symposium on Integrated Network Management, Asia-Pacific Network Operations and Management Symposium, End-to-End Monitoring Techniques and Services (E2EMON), IEEE Consumer Communications and Networking Conference, Assurance in Distributed Systems and Networks, International Conference on Parallel Processing, Data Integration and Mining, World Conference on Information Security Applications, Broadband Convergence Network, Telecommunication Information Networking Architecture, International Symposium on Applications and the Internet, and International Conference on Information Networking. He was an Associate Editor of the IEEE TRANSACTIONS ON NETWORK AND SERVICE MANAGEMENT, the IEEE JOURNAL OF COMMUNICATIONS AND NETWORKS, and the *International Journal of Network Management*, and an Associate Technical Editor of the *IEEE Communications Magazine*. He currently serves as an Associate Editor for the *International Journal of Network Management* and *Future Internet*. He is a member of the Association for Computing Machinery, the Institute of Electronics, Information and Communication Engineers, the Information Processing Society of Japan, the Korean Institute of Information Scientists and Engineers, the Korean Institute of Communications and Information Sciences, the Korean Information Processing Society, and the Open Standards and ICT Association.



Huy Q. Le received the B.E. degree in control engineering automation from the Ho Chi Minh City University of Technology, Ho Chi Minh City, Vietnam, in 2019. He is currently pursuing the Ph.D. degree in computer science with Kyung Hee University, South Korea. His research interests include machine learning, federated learning, self-supervised learning, and multimodal learning.



Chu Myaet Thwal received the B.E. degree in computer engineering and information technology from Yangon Technological University, Myanmar, in 2019. She is currently pursuing the Ph.D. degree in computer science and engineering with Kyung Hee University, South Korea. Her research interests include deep learning, artificial intelligence, and federated learning.